行动胜于语言：面向生成式推荐的万亿参数序列转导器

Jiaqi Zhai [1] **Lucy Liao** [1] **Xing Liu** [1] **Yueming Wang** [1] **Rui Li** [1]
Xuan Cao [1] **Leon Gao** [1] **Zhaojie Gong** [1] **Fangda Gu** [1] **Michael He** [1] **Yinghai Lu** [1] **Yu Shi** [1]

Abstract

大规模推荐系统依赖高基数的异构特征，并且每天需要处理数百亿次用户行为。尽管使用数千个特征和海量数据进行训练，工业界大多数深度学习推荐模型（DLRM）的效果仍无法随计算量扩展。受 Transformer 在语言和视觉领域取得成功的启发，我们重新审视推荐系统的基本设计选择。我们在生成式建模框架（“生成式推荐器”）下，将推荐问题重新表述为序列转导任务，并提出了面向高基数、非平稳流式推荐数据的新架构 HSTU。HSTU 在合成数据集和公开数据集上相较基线的 NDCG 提升最高达 65.8%，在长度为 8192 的序列上，其速度是基于 FlashAttention2 的 Transformer 的 5.3 至 15.2 倍。基于 HSTU、拥有 1.5 万亿参数的生成式推荐器在在线 A/B 测试中将指标提升了 12.4%，并已部署于拥有数十亿用户的大型互联网平台的多个业务场景。更重要的是，实证结果表明，生成式推荐器的模型质量随训练计算量呈幂律增长，覆盖三个数量级，达到 GPT-3/LLaMa-2 的规模。这减少了未来模型开发所需的碳足迹，并进一步为推荐领域首批基础模型铺平了道路。

1. 引言

推荐系统是在线内容平台和电子商务的核心，每天为数十亿用户提供个性化体验，发挥着关键作用。推荐领域最先进的方法一直基于深度学习推荐模型（DLRM）(Mudigere et al., 2022)，这一趋势已延续近十年 (Covington et al., 2016; Cheng et al., 2016; Zhou et al., 2018; Tang et al., 2020; Wang et al., 2021; Xia et al., 2023)。DLRM 的特点是使用异构特征，例如计数器和比率等数值特征、嵌入，以及创作者 ID、用户 ID 等类别特征。由于每分钟都有新内容和新商品加入，特征空间的基数极高，通常达到数十亿量级 (Eksombatchai et al., 2018)。为利用数以万计的此类特征，DLRM 使用各种神经网络来组合特征、变换中间表示，并形成最终输出。

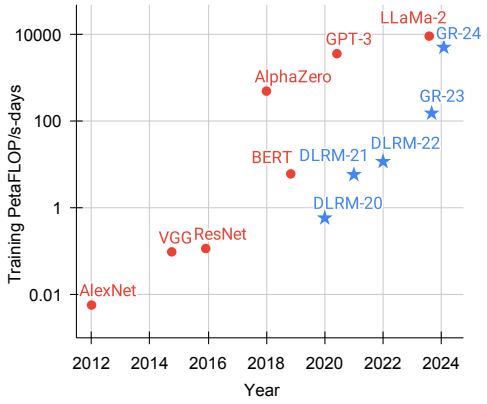


Figure 1. 历年来训练深度学习模型所使用的总计算量。DLRM 的结果来自 (Mudigere et al., 2022)；GR 为本文已部署的模型。DLRM/GR 在流式设置下持续训练；我们报告每年的计算量。

尽管使用了大量人工设计的特征并在海量数据上训练，工业界大多数 DLRM 的效果随计算量扩展的能力仍然较差 (Zhao et al., 2023)。这一局限值得关注，且其原因仍未得到解答。

受 Transformer 在语言和视觉领域取得成功的启发，我们重新审视现代推荐系统的基本设计选择。我们发现，要在数十亿用户规模下采用其他问题表述方式，必须克服三个挑战。首先，推荐系统中的特征缺乏显式结构。虽然已有研究在小规模设置下探索序列化表述（详细讨论见 Appendix B），但高基数 ID、交叉特征、计数器、比率等异构特征在工业级 DLRM 中发挥着关键作用 (Mudigere et al., 2022)。其次，推荐系统使用持续变化的十亿级词表。与语言领域十万级的静态词表 (Brown et al., 2020) 相比，十亿级动态词表给训练带来了挑战；而由于需要以目标感知的方式考虑数以万计的候选项，推理成本也很高 (Zhou et al., 2018; Wang et al., 2020)。最后，计算成本是实现大规模序列模型的主要瓶颈。GPT-3 使用数千块 GPU，在 1–2 个月内总计训练了 3000 亿个 token (Brown et al., 2020)。这一规模看似惊人，但与用户行为的规模相比则并非如此。最大的互联网平台服务于数十亿日活跃用户，这些用户每天与数十亿条帖子、图片和视频互动。用户序列长度可达 10^5 (Chang et al., 2023)。因此，推荐系统每天需要处理的 token 数，比语言模型在 1–2 个月内处理的数量还要高几个数量级。

本文将*用户行为*视为生成式建模中的一种新*模态*。我们的核心洞见是：a）在适当的新特征空间下，工业级

[1]MRS, Meta AI. Correspondence to: <{jiaqiz, lucyyl, xingl, yuemingw, ruili}@meta.com>. Code available at https://github.com/facebookresearch/generative-recommenders.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024. Copyright 2024 by the author(s).

推荐器中的核心排序和召回任务可以表述为生成式建模问题；b）这一范式使我们能够系统地利用特征、训练和推理中的冗余来提高效率。借助这一新表述，我们部署了计算复杂度比此前最先进方法高三个*数量级*的模型，同时将核心业务指标提升了 12.4%，如 Figure 1 所示。

我们的贡献如下。首先，我们在 Section 2 中提出*生成式推荐器*（GR），这是一种替代 DLRM 的新范式。我们将 DLRM 中的异构特征空间序列化并统一起来；随着序列长度趋于无穷，新方法能够逼近完整的 DLRM 特征空间。这使我们可以将排序和召回这两个主要推荐问题重新表述为 GR 中纯粹的序列转导任务。更重要的是，这进一步使模型能够以序列化、生成式的方式训练，从而允许我们*使用相同的计算量*在多出数个数量级的数据上训练。

随后，我们解决训练和推理全过程中的计算成本挑战。我们提出一种新的序列转导架构——*层次化序列转导单元*（HSTU）。HSTU 针对大规模、非平稳词表修改了注意力机制，并利用推荐数据集的特点，在长度为 8192 的序列上实现了相较基于 FlashAttention2 的 Transformer 的 5.3 至 15.2 倍加速。此外，通过利用微批处理充分摊销计算成本的新算法 M-FALCON（Section 3.4），我们能够在线服务*复杂度为 285 倍*的 GR 模型，同时实现 1.50–2.99 倍加速，且*均使用与传统 DLRM 相同的推理预算*。

最后，我们在 Section 4 中通过合成数据集、公开数据集，以及在拥有数十亿日活跃用户的大型互联网平台多个业务场景中的部署，验证了所提出的技术。据我们所知，本文首次表明，在大规模工业场景中，HSTU 这类纯序列转导架构在生成式设置（GR）下能够显著优于 DLRM。值得关注的是，我们不仅克服了传统 DLRM 已知的扩展瓶颈，还成功表明缩放定律 (Kaplan et al., 2020) 适用于推荐，这可能意味着推荐系统的 ChatGPT 时刻即将到来。

2. 将推荐表述为序列转导任务：从 DLRM 到 GR

2.1. 统一 DLRM 中的异构特征空间

现代 DLRM 通常使用大量类别（“稀疏”）特征和数值（“稠密”）特征进行训练。在 GR 中，我们将这些特征整合并编码为一个统一的时间序列，如 Figure 2 所示。

类别（“稀疏”）特征。此类特征包括用户点赞的物品、用户关注的某一类别（例如户外）的创作者、用户语言、用户加入的社区、请求发起城市等。我们按以下方式将这些特征*序列化*。首先选择最长的时间序列作为主时间序列，通常通过合并表示用户曾互动的物品的特征来获得。其余特征通常是随时间缓慢变化的时间序列，例如人口统计属性或关注的创作者。我们对这些时间序列进行*压缩*：每个连续片段仅保留最早的条目，再将结果合并到主时间序列中。由于这些时间序列变化很慢，这一方法不会显著增加总序列长度。

数值（“稠密”）特征。此类特征包括加权和衰减计数器、比率等。例如，某个特征可以表示用户过去对某一主题下物品的点击率（CTR）。与类别特征相比，这些特征变化频繁得多，可能在每次（用户，物品）交互后都发生变化。因此，从计算和存储角度看，将此类特征完全序列化并不可行。不过，一个重要观察是：执行这些聚合所依赖的类别特征（例如物品主题、位置）已经在 GR 中进行了序列化和编码。因此，我们可以在 GR 中移除数值特征，因为随着 GR 的总序列长度和计算量增加，具有足够表达能力的序列转导架构*结合目标感知的问题表述* (Zhou et al., 2018)，能够有效捕获数值特征。

2.2. 将排序和召回重新表述为序列转导任务

给定按时间顺序排列的 n 个 token $x_0, x_1, \ldots, x_{n-1}$（$x_i \in \mathbb{X}$），以及观测到这些 token 的时刻 $t_0, t_1, \ldots, t_{n-1}$，序列转导任务将该输入序列映射为输出 token $y_0, y_1, \ldots, y_{n-1}$（$y_i \in \mathbb{X} \cup \{\varnothing\}$），其中 $y_i = \varnothing$ 表示 y_i 未定义。

我们用 $\Phi_i \in \mathbb{X}_c$（$\mathbb{X}_c \subseteq \mathbb{X}$）表示系统向用户提供的一项内容（例如图片或视频）。由于新内容不断被创建，$\mathbb{X}_c$ 和 $\mathbb{X}$ 都是非平稳的。用户可以对 Φ_i 作出某种行为 a_i（例如点赞、跳过、完播并分享）作为回应，其中 $a_i \in \mathbb{X}$。我们用 n_c 表示用户交互过的内容总数。

于是，在因果自回归设置下，标准排序和召回任务可以定义为序列转导任务（Table 1）。我们有以下观察：

Task		Specification (Inputs / Outputs)
Ranking	x_is	$\Phi_0, a_0, \Phi_1, a_1, \ldots, \Phi_{n_c-1}, a_{n_c-1}$
	y_is	$a_0, \varnothing, a_1, \varnothing, \ldots, a_{n_c-1}, \varnothing$
Retrieval	x_is	$(\Phi_0, a_0), (\Phi_1, a_1), \ldots, (\Phi_{n_c-1}, a_{n_c-1})$
	y_is	$\Phi'_1, \Phi'_2, \ldots, \Phi'_{n_c-1}, \varnothing$ ($\Phi'_i = \Phi_i$ if a_i is positive, otherwise $\varnothing$)

Table 1. 作为序列转导任务的排序和召回。为简洁起见，省略其他类别特征。我们在 Appendix B.2 中比较 GR 与传统序列推荐器。

召回。在推荐系统的召回阶段，我们学习分布 $p(\Phi_{i+1}|u_i)$，其定义在 $\Phi_{i+1} \in \mathbb{X}_c$ 上，其中 u_i 是用户在 token i 处的表示。一个典型目标是选择 $\arg\max_{\Phi \in \mathbb{X}_c} p(\Phi|u_i)$，以最大化某种奖励。这与标准自回归设置有两点不同。首先，x_i 的监督目标 y_i 不一定是 Φ_{i+1}，因为用户可能对 Φ_{i+1} 作出负面回应。其次，y_i 在 x_{i+1} 表示人口统计属性等与互动无关的类别特征时未定义。

排序。GR 中的排序任务面临独特挑战，因为工业推荐系统通常需要“目标感知”的问题表述。在这种设置下，目标 Φ_{i+1} 与历史特征的“交互”需要尽早发生；而标准自回归设置的“交互”发生得较晚（例如在编码器输出后的 softmax 中），因而无法实现这一点。我们通过交*错排列*物品和行为来解决这一问题，如 Table 1 所示，从而可以将排序任务表述为 $p(a_{i+1}|\Phi_0, a_0, \Phi_1, a_1, \ldots, \Phi_{i+1})$（尚未加入类别特征）。实践中，我们使用一个小型神经

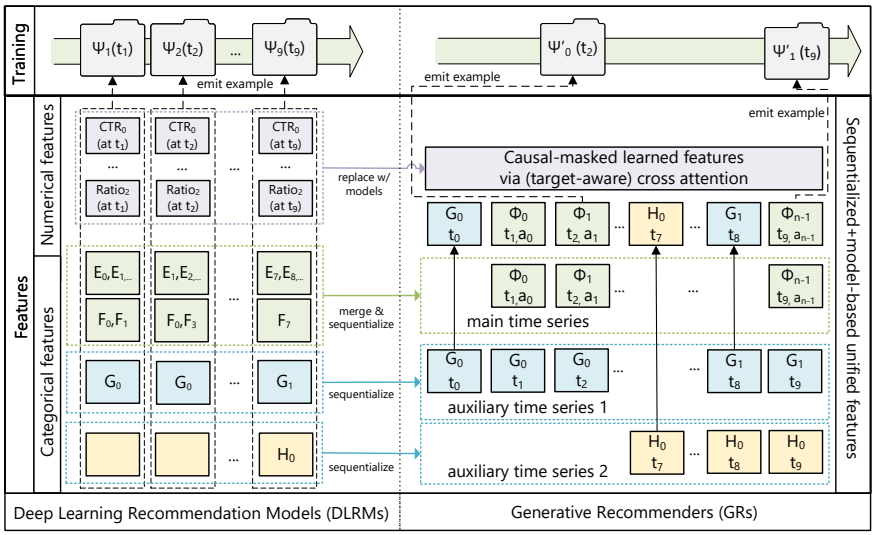


Figure 2. 特征和训练流程对比：DLRM 与 GR。E, F, G, H 表示类别特征。Φ_i 表示合并后的主时间序列中的第 i 个物品。$\Psi_k(t_j)$ 表示第 k 个训练样本，其输出时刻为t_j。完整符号说明见 Appendix A。

网络，将 Φ_{i+1} 处的输出转换为多任务预测。更重要的是，这使我们能够在一次前向处理中，对全部 n_c 次互动应用目标感知交叉注意力。

2.3. 生成式训练

工业推荐器通常在流式设置下训练，每个样本产生后按顺序处理。在这种设置下，Transformer (Vaswani et al., 2017) 等基于自注意力的序列转导架构的总计算需求随 $\sum_i n_i(n_i^2 d + n_i d_{ff} d)$ 扩展，其中 n_i 是用户 i 的 token 数，d 为嵌入维度。括号中的第一部分来自自注意力；这里假设其扩展因子为 $O(n^2)$，因为大多数亚二次算法需要在质量上作出取舍，且实际运行时间劣于二次算法 (Dao et al., 2022)。第二部分来自逐点 MLP 层，其隐藏层大小为 $O(d_{ff}) = O(d)$。令 $N = \max_i n_i$，总时间复杂度化为 $O(N^3 d + N^2 d^2)$，这一成本在推荐场景中难以承受。

为解决在长序列上可扩展地训练序列转导模型的挑战，我们从传统的曝光级训练转向*生成式训练*，将计算复杂度降低 $O(N)$ 倍，如图 2 顶部所示。这样，编码器成本便可在多个目标之间摊销。具体而言，当第 i 个用户的采样率为 $s_u(n_i)$ 时，总训练成本随 $\sum_i s_u(n_i) n_i (n_i^2 d + n_i d^2)$ 扩展；该成本可降至 $O(N^2 d + N d^2)$，只需将 $s_u(n_i)$ 设为 $1/n_i$。在工业级系统中实现这一采样的一种方式，是在用户请求或会话结束时输出训练样本，从而得到 $\hat{s_u}(n_i) \propto 1/n_i$。

3. 面向生成式推荐的高性能自注意力编码器

为将 GR 扩展到具有大规模、非平稳词表的工业级推荐系统，我们接下来介绍一种新的编码器设计——*层次化序列转导单元*（HSTU）。HSTU 由多个相同的层堆叠而成，层间通过残差连接 (He et al., 2015) 相连。每一层包含三个子层：逐点投影（式 1）、空间聚合（式 2）以及逐点变换（式 3）：

$$U(X), V(X), Q(X), K(X) = \text{Split}(\phi_1(f_1(X))) \quad (1)$$

$$A(X)V(X) = \phi_2\left(Q(X)K(X)^T + \text{rab}^{p,t}\right)V(X) \quad (2)$$

$$Y(X) = f_2\left(\text{Norm}\left(A(X)V(X)\right) \odot U(X)\right) \quad (3)$$

其中 $f_i(X)$ 表示 MLP；我们使用单个线性层 $f_i(X) = W_i(X) + b_i$ 实现 f_1 和 f_2，以降低计算复杂度，并通过融合内核进一步批量计算查询 $Q(X)$、键 $K(X)$、值 $V(X)$ 和门控权重 $U(X)$；ϕ_1 和 ϕ_2 表示非线性函数，两者均采用 SiLU (Elfwing et al., 2017)；Norm 为层归一化；$\text{rab}^{p,t}$ 表示结合位置（p）和时间（t）信息的相对注意力偏置 (Raffel et al., 2020)。所用符号的完整说明见 Table 9。

HSTU 编码器的设计允许用单一的模块化结构块替代 DLRM 中的异构模块。我们观察到，DLRM 实际上有三个主要阶段：*特征提取*、*特征交互*和*表示变换*。*特征提取*获取类别特征经过池化后的嵌入表示。最先进的此类方法可以概括为成对注意力与目标感知池化 (Zhou et al., 2018)，而 HSTU 层能够涵盖这些功能。

*特征交互*是 DLRM 最关键的部分。常用方法包括因子分解机及其神经网络变体 (Rendle, 2010; Guo et al., 2017; Xiao et al., 2017)、高阶特征交互 (Wang et al., 2021) 等。HSTU 通过 $\text{Norm}\left(A(X)V(X)\right) \odot U(X)$ 让经过注意力池化的特征与其他特征直接“交互”，从而替代特征交互模块。该设计源于使用学习到的 MLP 逼近点积的困难 (Rendle et al., 2020; Zhai et al., 2023a)。由于对 $U(X)$ 应用了 SiLU，$\text{Norm}\left(A(X)V(X)\right) \odot U(X)$ 也可以解释为 SwiGLU (Shazeer, 2020) 的一种变体。

*表示变换*通常通过混合专家（MoE）和路由完成，以处理多样化的异构人群。其核心思想是为不同用户专门设置子网络，从而执行条件计算 (Ma et al., 2018; Tang et al., 2020)。除一个归一化因子外，HSTU 中的逐元素乘积实际上能够实现 MoE 所使用的门控操作。

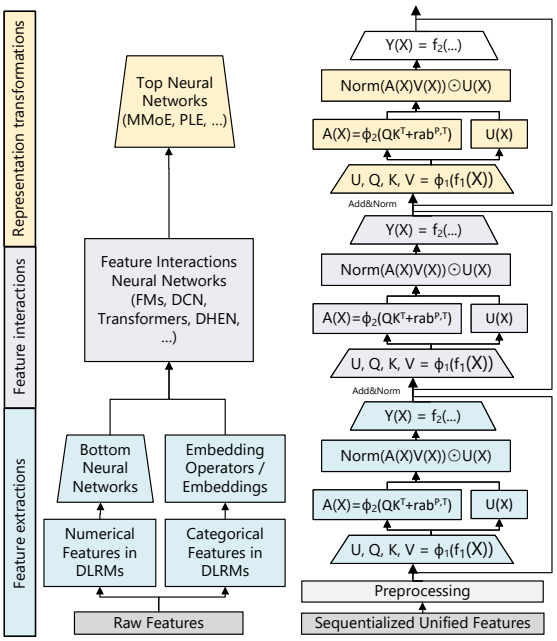


Figure 3. 关键模型组件对比：DLRM 与 GR。左侧为完整的 DLRM 配置 (Mudigere et al., 2022)，右侧为简化的 HSTU。

Architecture	HR@10	HR@50
Transformers	.0442	.2025
HSTU (-rab^{p,t}, Softmax)	.0617	.2496
HSTU (-rab^{p,t})	**.0893**	**.3170**

Table 2. 单遍流式设置下的合成数据。

3.1. 逐点聚合注意力

HSTU 采用一种新的逐点聚合（并归一化）注意力机制（相比之下，softmax 注意力在整个序列上计算归一化因子）。这一设计基于两个因素。首先，与目标相关的历史数据点数量是反映用户偏好强度的重要特征，但经过 softmax 归一化后便难以捕获。这一点很关键，因为我们既要预测互动强度，例如用户在给定物品上花费的时间，也要预测物品的相对顺序，例如预测使 AUC 最大化的排序。其次，尽管 softmax 激活在设计上对噪声具有鲁棒性，但它不太适用于流式设置中的非平稳词表。

所提出的逐点聚合注意力机制如 Equation (2) 所示。重要的是，逐点池化之后需要使用层归一化，以稳定训练。理解这一设计的一种方式，是使用遵循狄利克雷过程的合成数据，该过程在非平稳词表上生成流式数据（详见附录 C）。在这一设置中，我们可以观察到 softmax 与逐点注意力设置之间高达 44.7% 的差距，如 Table 2 所示。

3.2. 利用稀疏性并通过算法提高稀疏性

在推荐系统中，用户历史序列的长度通常服从偏斜分布，因此输入序列具有稀疏性，尤其是在序列很长的设置下。利用这种稀疏性，可以显著提高编码器效率。为此，我们为 GPU 开发了高效注意力内核，以类似 (Rabe & Staats, 2021; Dao et al., 2022) 的方式融合连续的 GEMM，但执行*完全基于不规则张量*的注意力计算。这实际上将注意力计算转化为一组不同尺寸的分组 GEMM（Appendix G）。因此，HSTU 的自注意力变为受内存访问限制，其内存访问量按 $\Theta(\sum_i n_i^2 d_{qk}^2 R^{-1})$ 扩展，其中 n_i 是样本 i 的序列长度，d_{qk} 为注意力维度，R 为寄存器大小。仅这一方法就能带来 2–5 倍吞吐量增益，详见 Section 4.2。

Alpha (α)	Max Sequence Lengths			
	1,024	2,048	4,096	8,192
1.6	71.5%	76.1%	80.5%	84.4%
1.7	56.1%	63.6%	69.8%	75.6%
1.8	40.2%	45.3%	54.1%	66.4%
1.9	17.2%	21.0%	36.3%	64.1%
2.0	3.1%	6.6%	29.1%	64.1%

Table 3. *随机长度*（SL）对序列稀疏率的影响。

我们进一步通过*随机长度*（SL）从*算法上*提高用户历史序列的稀疏性。推荐场景中用户历史序列的一个关键特点是，用户行为在时间上具有重复性，因为用户行为会在整个交互历史的多个尺度上呈现。这使我们有机会在不损害模型质量的情况下人为提高稀疏性，从而显著降低按 $\Theta(\sum_i n_i^2)$ 扩展的编码器成本。

我们可以将用户 j 的历史表示为序列 $(x_i)_{i=0}^{n_{c,j}}$，其中 $n_{c,j}$ 是用户交互过的内容数。令 $N_c = \max_j n_{c,j}$。令 $(x_{i_k})_{k=0}^{L}$ 为长度为 L、从原始序列 $(x_i)_{i=0}^{n_{c,j}}$ 构造的子序列。SL 按如下方式选择输入序列：

$$(x_i)_{i=0}^{n_{c,j}} \text{ if } n_{c,j} \le N_c^{\alpha/2}$$
$$(x_{i_k})_{k=0}^{N_c^{\alpha/2}} \text{ if } n_{c,j} > N_c^{\alpha/2}, \text{w/ probability } 1 - N_c^{\alpha}/n_{c,j}^2 \quad (4)$$
$$(x_i)_{i=0}^{n_{c,j}} \text{ if } n_{c,j} > N_c^{\alpha/2}, \text{w/ probability } N_c^{\alpha}/n_{c,j}^2$$

这将与注意力相关的复杂度降至 $O(N_c^{\alpha} d) = O(N^{\alpha} d)$，其中 $\alpha \in (1, 2]$。关于子序列选择的更详细讨论见 Appendix F.1。我们指出，将 SL 应用于训练能够形成具有成本效益的系统设计，因为训练的计算成本通常显著高于推理。

Table 3 展示了一个使用 30 天用户历史的代表性工业级配置中，不同序列长度和 α 值对应的*稀疏率*（见 Appendix F）。模型质量退化可忽略的配置以下划线和蓝色突出显示。标记为“$\alpha = 2.0$”的行表示未应用 SL 的基准稀疏率情况。较低的 α 适用于较长的序列，直至我们测试的最长序列长度 8,192。

3.3. 最小化激活内存占用

在推荐系统中，使用大批量对于训练吞吐量 (Mudigere et al., 2022) 和模型质量 (Yang et al., 2020; Chen et al., 2020; Zhai et al., 2023a) 都至关重要。因此，激活内存占用成为主要的扩展瓶颈；相比之下，大语言模型通常使用小批量训练，内存占用则以参数为主。

与 Transformer 相比，HSTU 采用简化且充分融合的设计，显著降低了激活内存占用。首先，HSTU 将注意力之外的线性层数从六层减少为两层，这与近期使

用逐元素门控减少 MLP 计算的工作 (Hua et al., 2022; Gu et al., 2022) 一致。其次，HSTU 尽可能将计算融合为单个算子，包括 Equation (1) 中的 $\phi_1(f_1(\cdot))$，以及 Equation (3) 中的层归一化、可选 dropout 和输出 MLP。这一简化设计将 bfloat16 精度下每层的激活内存占用降低至 $2d + 2d + 4hd_{qk} + 4hd_v + 2hd_v = 14d$。

作为对比，Transformer 在注意力之后使用前馈层和 dropout（中间状态为 $3hd_v$），随后使用由层归一化、线性层、激活函数、线性层和 dropout 组成的逐点前馈块，其中间状态为 $2d + 4d_{ff} + 2d + 1d = 4d + 4d_{ff}$。这里采用标准假设：$hd_v \geq d$ 且 $d_{ff} = 4d$ (Vaswani et al., 2017; Brown et al., 2020)。因此，计入输入、输入层归一化（$4d$）以及 qkv 投影后，总激活状态量为 $33d$。由此，HSTU 的设计支持将网络深度扩展到 > 2 倍。

此外，用于表示词表的大规模原子 ID 也需要大量内存。对于 100 亿大小的词表、512 维嵌入和 Adam 优化器，仅以 fp32 存储嵌入和优化器状态就需要 60TB 内存。为缓解内存压力，我们采用逐行 AdamW 优化器 (Gupta et al., 2014; Khudia et al., 2021)，并将优化器状态放在 DRAM 中，从而将每个浮点数的 HBM 占用从 12 字节降低到 2 字节。

3.4. 通过成本摊销扩展推理

我们解决的最后一个挑战是，推荐系统在在线服务时需要处理大量候选项。我们重点关注排序，因为对于召回，编码器成本能够被充分摊销；无论是利用量化、哈希或分区的最大内积搜索（MIPS） (Jegou et al., 2011; Shrivastava & Li, 2014; Li et al., 2002; Zhai et al., 2011)，还是通过束搜索或层次化召回处理非 MIPS 情况 (Zhuo et al., 2020; Zhai et al., 2023a)，都已有高效算法。

对于排序，我们需要处理多达数以万计的候选项 (Covington et al., 2016; Wang et al., 2020)。我们提出 M-FALCON 算法（利用可缓存运算的微批次快速注意力），以对 m 个候选项执行推理，输入序列长度为 n。

在一次前向处理中，M-FALCON 并行处理 b_m 个候选项，其方式是修改注意力掩码和 $\text{rab}^{p,t}$ 偏置，使对这 b_m 个候选项执行的注意力操作完全相同。这将应用交叉注意力的成本从 $O(b_m n^2 d)$ 降至 $O((n + b_m)^2 d) = O(n^2 d)$，条件是 b_m 相对于 n 可视为一个小常数。我们还可以将总共 m 个候选项划分为 $\lceil m/b_m \rceil$ 个大小为 b_m 的微批次，以利用编码器层面的 KV 缓存 (Pope et al., 2022)：既可以在多次前向处理之间复用以降低成本，也可以跨*请求*复用以尽量减少尾延迟（更详细的讨论见 Appendix H）。

总体而言，M-FALCON 使模型复杂度能够随传统 DLRM 排序阶段的候选数量线性扩展；我们成功应用了复杂度为 *285 倍*的目标感知交叉注意力模型，并实现了 1.5–3 倍吞吐量，且在 Section 4.3 讨论的典型排序配置下保持推理预算不变。

4. 实验

4.1. 验证 HSTU 编码器的归纳假设

4.1.1. 传统序列设置

我们首先在两个常用推荐数据集 MovieLens 和 Amazon Reviews 上评估 HSTU 的性能。我们遵循文献中的序列推荐设置，包括*完全打乱*和*多轮*训练。基线采用先进的 Transformer 实现 SASRec (Kang & McAuley, 2018) [1]。与近期工作 (Dallmann et al., 2021; Zhai et al., 2023a) 一致，我们在整个物品集合上报告 Hit Rate@K 和 NDCG@K。

结果见 Table 4。“SASRec (2023)”表示 (Zhai et al., 2023a) 报告的最佳 SASRec 训练方案。标记为“HSTU”的行使用与 SASRec 相同的配置（层数、注意力头数等均相同）。“HSTU-large”表示更大的 HSTU 编码器（层数为 4 倍，注意力头数为 2 倍）。结果表明：a）HSTU 的设计针对推荐进行了优化，在相同配置下显著优于基线；b）扩展 HSTU 的规模能进一步提升性能。

需要强调，这里的评估方法与工业级设置有显著区别，因为*完全打乱*和*多轮*训练在工业界使用的流式设置中通常不可行 (Liu et al., 2022)。

4.1.2. 工业级流式设置

接下来，我们使用工业级数据集，在流式设置下比较 HSTU、HSTU 的消融版本与 Transformer 的性能。在本节其余部分，我们为排序报告归一化熵（NE） (He et al., 2014)。我们使用 1000 亿个样本（按 DLRM 等效样本计）训练模型，每个训练任务使用 64–256 块 H100。由于排序采用多任务设置，我们报告主要互动事件（“E-Task”）和主要消费事件（“C-Task”）的结果。在我们的场景中，NE 降低 0.001 被视为显著改善，因为它通常会为数十亿用户带来 0.5% 的核心业务指标提升。对于召回，由于设置与语言建模类似，我们报告对数困惑度。受资源限制，我们将编码器参数固定在较小规模的设置（排序采用 $l = 3$、$n = 2048$、$d = 512$，召回采用 $l = 6$、$n = 512$、$d = 256$），并对其他超参数进行网格搜索。

结果见 Table 5。首先，HSTU 显著优于 Transformer，尤其是在排序中，这可能源于逐点注意力和改进的相对注意力偏置。其次，HSTU 消融版本与 HSTU 之间的差距验证了各项设计的有效性。由于训练稳定性的限制，基于 Softmax 的 HSTU 和 Transformer 的最优学习率约为其他模型的十分之一。即使采用更低的学习率和前置归一化残差连接 (Xiong et al., 2020)，我们在排序中使用标准 Transformer 时仍频繁遇到损失爆炸。最后，HSTU 优于 LLM 中常用的 Transformer 变体 Transformer++ (Touvron et al., 2023a)，后者使用 RoPE (Su et al., 2023)、SwiGLU 等技术。总体而言，在这一小规模设置下，HSTU 在质量更好的同时，实际运行速度为对照的 1.5–2 倍，HBM 占用减少 50%。

[1] 其他基线的结果见 Appendix D。

Table 4. 在多遍、完全打乱设置下，各方法在公开数据集上的评估。

	Method	HR@10	HR@50	HR@200	NDCG@10	NDCG@200
ML-1M	SASRec (2023)	.2853	.5474	.7528	.1603	.2498
	HSTU	.3097 (+8.6%)	.5754 (+5.1%)	.7716 (+2.5%)	.1720 (+7.3%)	.2606 (+4.3%)
	HSTU-large	**.3294 (+15.5%)**	**.5935 (+8.4%)**	**.7839 (+4.1%)**	**.1893 (+18.1%)**	**.2771 (+10.9%)**
ML-20M	SASRec (2023)	.2906	.5499	.7655	.1621	.2521
	HSTU	.3252 (+11.9%)	.5885 (+7.0%)	.7943 (+3.8%)	.1878 (+15.9%)	.2774 (+10.0%)
	HSTU-large	**.3567 (+22.8%)**	**.6149 (+11.8%)**	**.8076 (+5.5%)**	**.2106 (+30.0%)**	**.2971 (+17.9%)**
Books	SASRec (2023)	.0292	.0729	.1400	.0156	.0350
	HSTU	.0404 (+38.4%)	.0943 (+29.5%)	.1710 (+22.1%)	.0219 (+40.6%)	.0450 (+28.6%)
	HSTU-large	**.0469 (+60.6%)**	**.1066 (+46.2%)**	**.1876 (+33.9%)**	**.0257 (+65.8%)**	**.0508 (+45.1%)**

Table 5. 在单遍流式设置下，HSTU、HSTU 消融版本和 Transformer 在工业级数据集上的评估。

Architecture	Retrieval	Ranking (NE)	
	log pplx.	E-Task	C-Task
Transformers	4.069	NaN	NaN
HSTU (-rab^{p,t}, Softmax)	4.024	.5067	.7931
HSTU (-rab^{p,t})	4.021	.4980	.7860
Transformer++	4.015	.4945	.7822
HSTU (original rab)	4.029	.4941	.7817
HSTU	**3.978**	**.4937**	**.7805**

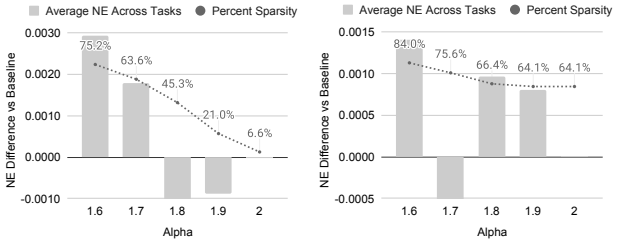


Figure 4. 随机长度（SL）对指标的影响。左：$n = 4096$。右：$n = 8192$。完整结果见 Appendix F。

4.2. 编码器效率

随机长度。Figure 4 和 Figure 5（a）展示了*随机长度*（SL）对模型指标的影响。当 $\alpha = 1.6$ 时，长度为 4096 的序列在大多数时候被转化为长度为 776 的序列，即移除超过 80% 的 token。即使稀疏率增加到 64%–84%，主要任务的 NE 退化也未超过 0.002（0.2%）。这些证据表明，当 α 取值合适时，SL 不会对模型质量产生负面影响，并且可以通过高稀疏率降低训练成本。我们还在 Appendix F.3 中验证了 SL 显著优于现有长度外推技术。

编码器效率。Figure 5 比较了 HSTU 与 Transformer 编码器在训练和推理设置下的效率。对于 Transformer，我们采用先进的 FlashAttention-2 (Dao, 2023) 实现。我们考虑 1,024 至 8,192 的序列长度，并在训练中应用*随机长度*（SL）。评估时，HSTU 和 Transformer 使用相同配置（$d = 512$、$h = 8$、$d_{qk} = 64$），并移除*相对注意力偏置*，因为 Section 4.1.2 已表明，即使不使用 rab^{p,t}，HSTU 仍优于 Transformer。我们在 NVIDIA H100 GPU 上以 bfloat16 精度比较编码器层面的性能。总体而言，HSTU 在训练和推理中的效率最高分别为 Transformer 的 15.2 倍和 5.6 倍。

Table 6. 召回模型的离线/在线比较。

Methods	Offline HR@K		Online metrics	
	K=100	K=500	E-Task	C-Task
DLRM	29.0%	55.5%	+0%	+0%
DLRM (abl. features)	28.3%	54.3%	–	
GR (content-based)	11.6%	18.8%	–	
GR (interactions only)	35.6%	61.7%	–	
GR (new source)	**36.9%**	**62.4%**	**+6.2%**	**+5.0%**
GR (replace source)			**+5.1%**	**+1.9%**

Table 7. 排序模型的离线/在线比较。

Methods	Offline NEs		Online metrics	
	E-Task	C-Task	E-Task	C-Task
DLRM	.4982	.7842	+0%	+0%
DLRM (DIN+DCN)	.5053	.7899	–	–
DLRM (abl. features)	.5053	.7925	–	–
GR (interactions only)	.4851	.7903	–	–
GR	**.4845**	**.7645**	**+12.4%**	**+4.4%**

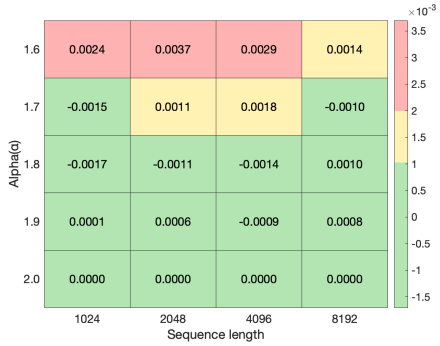
此外，第 3.3 节所讨论的激活内存占用降低，使我们能够用 HSTU 构建深度超过 Transformer 两倍的网络。

4.3. 工业级流式设置下生成式推荐器与 DLRM 的比较

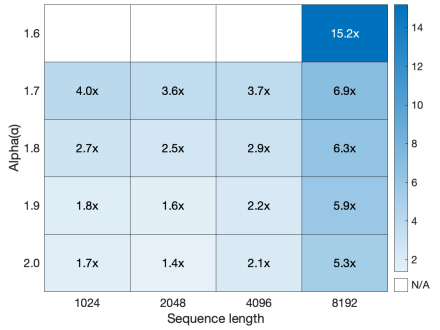
最后，我们在工业级流式设置下，将 GR 的端到端性能与先进的 DLRM 基线进行比较。我们的 GR 实现代表生产中使用的典型配置，而 DLRM 配置则凝聚了数百人多年的迭代成果。由于推荐系统的召回阶段使用多个生成器，我们同时报告添加 GR（“add source”）以及替换现有主要 DLRM 召回源（“replace source”）的在线结果。Table 6 和 Table 7 表明，GR 不仅在线下显著优于 DLRM，还在 A/B 测试中带来了 12.4% 的提升。

如 Section 2 所述，GR 基于原始类别互动特征构建，而 DLRM 通常使用多得多的特征训练，其中大部分是从原始信号中人工构造的。如果向 DLRM 提供 GR 所使用的同一组特征（“DLRM (abl. features)”），DLRM 的性能会显著退化。这说明 GR 能够通过其架构和统一特征空间有效捕获这些特征。

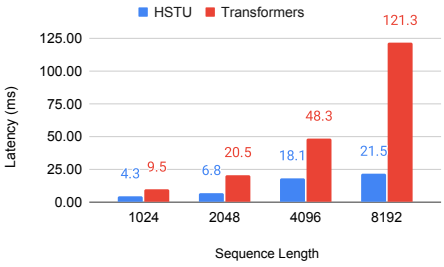
我们进一步将 Section 2.2 中的 GR 表述与仅考虑用户交互物品的传统序列推荐设置 (Kang & McAuley, 2018)（“GR (interactions only)”）进行比较，以验证 GR 的设计。后者结果明显更差，其排序版本在主要消费任务的 NE 上比 GR 差 2.6%。



(a) 训练 NE。



(b) 训练加速比。



(c) 推理加速比。

Figure 5. 编码器层面的效率：HSTU 与基于 FlashAttention2 的 Transformer 在训练（a、b）和推理（c）中的对比。

考虑到基于内容的方法（包括语言模型）的流行程度，我们还纳入了仅使用内容特征的 GR 基线（“GR (content-based)”）。内容基线与 DLRM/GR 之间显著的性能差距，凸显了高基数用户行为的重要性。

最后，我们在 Figure 6 中比较 GR 与生产 DLRM 的效率。尽管 GR 模型的计算复杂度为后者的 285 倍，在为 1024/16384 个候选项打分时，我们仍实现了后者 1.50/2.99 倍的更高 QPS，这得益于 HSTU 和 Section 3.4 中的新算法 M-FALCON。

4.3.1. 推荐系统的缩放定律

众所周知，在大规模工业场景中，DLRM 的质量在达到一定计算量和参数规模后会趋于饱和 (Zhao et al., 2023)。我们比较 GR 与 DLRM 的可扩展性，以更好地理解这一现象。

由于特征交互层对于 DLRM 的性能至关重要 (Mudigere et al., 2022)，我们尝试了 Transformer (Vaswani et al., 2017)、DHEN (Zhang et al., 2022)，以及生产配置中使用的 DCN (Wang et al., 2021) 变体（结合了残差连接 (He et al., 2015)），来扩展排序设置中的 DLRM 基线。对于召回基线，由于其采用残差结构，我们增大了隐藏层大小、嵌入维度和层数。对于基于 HSTU 的生成式推荐器（GR），我们通过调整 HSTU 超参数来扩展模型，包括残差层数、序列长度、嵌入维度、注意力头数等。对于召回，我们还调整了负样本数。

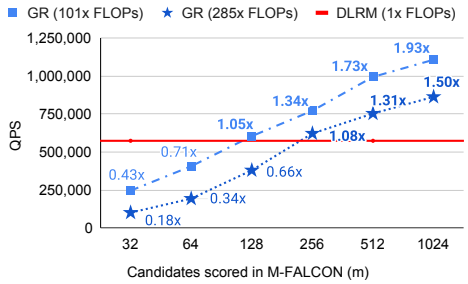


Figure 6. 最具挑战性的排序设置下的推理吞吐量比较。完整结果见 Appendix H.1。

结果见 Figure 7。在低计算量区域，DLRM 可能因人工设计的特征而优于 GR，这印证了特征工程对于传统 DLRM 的重要性。不过，GR 在 FLOPs 方面展现出明显更好的可扩展性，而 DLRM 的性能趋于平台期，这与既有工作的发现一致。我们还观察到，GR 在嵌入参数和非嵌入参数两方面都具有更好的可扩展性，使模型规模达到 1.5 万亿参数，而 DLRM 的性能在约 2000 亿参数时便趋于饱和。

最后，实证结果表明，在超参数合适的情况下，我们的全部主要指标，包括召回的 Hit Rate@100 和 Hit Rate@500 以及排序的 NE，均随所用计算量呈幂律变化。这一现象覆盖了三个数量级，直至我们能够测试的最大模型（序列长度 8,192、嵌入维度 1,024、24 层 HSTU）。此时，我们使用的总计算量（由于采用标准流式训练设置，按 365 天归一化）接近 GPT-3 (Brown et al., 2020) 和 LLaMa-2 (Touvron et al., 2023b) 的总训练计算量，如 Figure 1 所示。在合理范围内，与投入的总训练计算量相比，具体模型超参数的作用较小。与语言建模 (Kaplan et al., 2020) 不同，序列长度在 GR 中发挥着显著更重要的作用，因此必须同时扩大序列长度和其他参数。这或许是所提方法最重要的优势，因为我们首次表明，LLM 的缩放定律也可能适用于大规模推荐系统。

5. 相关工作

此前的序列推荐研究将用户交互简化为一个关于物品的同质序列 (Hidasi et al., 2016; Kang & McAuley, 2018)。序列方法在工业级场景中的应用，主要是成对注意力 (Zhou et al., 2018)，或作为 DLRM 组成部分的序列编码器 (Chen et al., 2019; Xia et al., 2023)。已有研究探索使用多阶段注意力替代自注意力，以提高效率 (Chang et al., 2023)。将 ID 表示为 token 序列的生成式方法，也已在召回中得到探索 (Zhuo et al., 2020)。我们在 Appendix B.1 中对既有工作作了更全面的讨论。

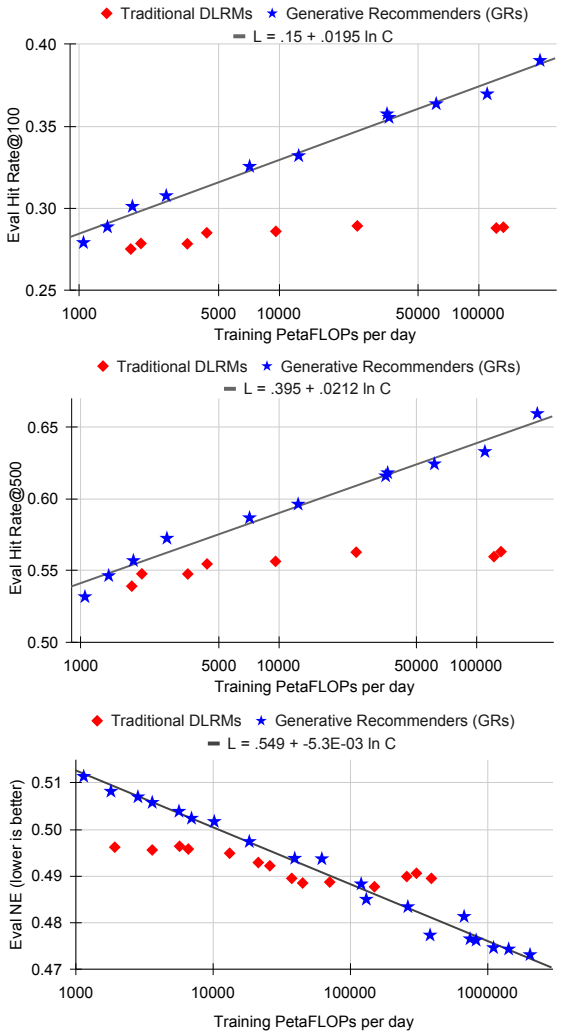


Figure 7. 可扩展性：大规模工业场景中，DLRM 与 GR 在召回（上、中）和排序（下）任务上的比较。HR 增加 0.005 和 NE 减少 0.001 均代表显著改善。

由于自注意力具有 $O(n^2)$ 的扩展因子，高效注意力一直是重要研究方向，主要工作包括因子化注意力 (Child et al., 2019)、低秩近似 (Katharopoulos et al., 2020) 等。近期，研究者还探索了序列转导设置的其他表述方式 (Gu et al., 2022; Hua et al., 2022)。其中，HSTU 的逐元素门控设计受到 FLASH (Hua et al., 2022) 的启发。近期的硬件感知设计已被证明能够显著降低内存占用 (Rabe & Staats, 2021; Korthikanti et al., 2022; Zhai et al., 2023b)，并显著改善实际运行时间 (Dao et al., 2022)。长度外推使在较短序列上训练的模型能够泛化，但大多数工作集中于微调或改进偏置机制 (Press et al., 2022)。我们的工作则受深度维度上随机性研究 (Huang et al., 2016) 的启发，在长度维度上引入随机性。

对大语言模型（LLM）的关注催生了一些工作，将各种推荐任务视为预训练 LLM 上的上下文学习 (Sileo et al., 2022)、指令微调 (Bao et al., 2023) 或迁移学习 (Li et al., 2023)。LLM 中蕴含的世界知识可以迁移到下游任务 (Cui et al., 2022)，并改善零样本或少样本场景中的推荐效果。用户行为序列的文本表示也已在中等规模数据集上展现出良好的扩展表现 (Shin et al., 2023)。大多数使用 LLM 进行推荐的研究集中于低数据量区域；在大规模设置下，它们在 MovieLens 上尚未优于协同过滤 (Hou et al., 2024)。

6. 结论

我们提出了生成式推荐器（GR）这一新范式，将排序和召回表述为序列转导任务，使其能够以生成式方式训练。实现这一点依赖于新型 HSTU 编码器设计，其在长度为 8192 的序列上的速度为先进 Transformer 的 5.3–15.2 倍，以及 M-FALCON 等新的训练和推理算法。借助 GR，我们部署了复杂度为 *285 倍*的模型，同时使用了*更少*的推理计算量。GR 和 HSTU 在生产环境中带来了 12.4% 的指标提升，并展现出优于传统 DLRM 的扩展性能。我们的结果印证了用户行为是生成式建模中一种尚未得到充分探索的模态——正如标题所言，*“行动胜于语言”*。

本文对特征的显著简化，使跨领域使用统一特征空间成为可能，从而为推荐、搜索和广告领域首批基础模型铺平道路。GR 完全序列化的设置还使推荐能够被表述为端到端的生成式任务。这两点都使推荐系统能够更全面地帮助用户。

影响声明

我们认为，本文具有广泛的积极影响。减少推荐、搜索和广告系统对大量异构特征的依赖，可以在改善用户体验的同时，使这些系统更加有利于隐私保护。通过完全序列化的表述，使推荐系统能够将用户的长期结果归因于短期决策，可以减少网络上不符合用户长期目标的内容（包括标题党和虚假新闻），并使平台激励与用户价值更好地一致。最后，基础模型和缩放定律的应用，有助于减少推荐、搜索及相关场景所需模型研究与开发带来的碳足迹。

致谢

这项工作凝聚了数百人的共同努力，以下贡献者的工作使其成为可能（按字母顺序排列）：Adnan Akhundov, Bugra Akyildiz, Shabab Ayub, Alex Bao, Renqin Cai, Jennifer Cao, Guoqiang Jerry Chen, Lei Chen, Sean Chen, Xianjie Chen, Huihui Cheng, Weiwei Chu, Ted Cui, Shiyan Deng, Nimit Desai, Fei Ding, Francois Fagan, Lu Fang, Liang Guo, Liz Guo, Jeevan Gyawali, Yuchen Hao, Daisy Shi He, Samuel Hsia, Jie Hua, Yanzun Huang, Hongyi Jia, Rui Jian, Jian Jin, Rahul Kindi, Changkyu Kim, Yejin Lee, Fu Li, Hong Li, Shen Li, Wei Li, Zhijing Li, Xueting Liao, Emma Lin, Hao Lin, Jingzhou Liu, Xingyu Liu, Kai Londenberg, Liang Luo, Linjian Ma, Matt Ma, Yun Mao, Bert Maher, Matthew Murphy, Satish Nadathur, Min Ni, Jongsoo Park, Jing Qian, Lijing Qin, Alex Singh, Timothy Shi, Dennis van der Staay, Xiao Sun, Colin Taylor, Shin-Yeh Tsai, Ro-

han Varma, Omkar Vichare, Alyssa Wang, Pengchao Wang, Shengzhi Wang, Wenting Wang, Xiaolong Wang, Zhiyong Wang, Wei Wei, Bin Wen, Carole-Jean Wu, Eric Xu, Bi Xue, Zheng Yan, Chao Yang, Junjie Yang, Zimeng Yang, Chunxing Yin, Daniel Yin, Yiling You, Keke Zhai, Yanli Zhao, Zhuoran Zhao, Hui Zhang, Jingjing Zhang, Lu Zhang, Lujia Zhang, Na Zhang, Rui Zhang, Xiong Zhang, Ying Zhang, Zhiyun Zhang, Charles Zheng, Erheng Zhong, Xin Zhuang. 我们感谢 Shikha Kapoor, Rex Cheung, Lana Dam, Ram Ramanathan, Nipun Mathur, Bo Feng, Yanhong Wu, Zhaohui Guo, Hongjie Bai, Wen-Yun Yang, Zellux Wang, Arun Singh, Bruce Deng, Yisong Song, Haotian Wu, Meihong Wang 提供的产品支持，并感谢 Joseph Laria, Akshay Hegde, Abha Jain, Raj Ganapathy 协助项目管理。最后，我们感谢 Ajit Mathews, Shilin Ding, Hong Yan, Lars Backstrom 提供的领导支持，以及与 Andrew Tulloch, Liang Xiong, Kaushik Veeraraghavan, 和 Gaofeng Zhao 进行的富有启发性的讨论。

References

Bao, K., Zhang, J., Zhang, Y., Wang, W., Feng, F., and He, X. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*, RecSys ' 23. ACM, September 2023. doi: 10.1145/3604915.3608857. URL http://dx.doi.org/10.1145/3604915.3608857.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. 2020.

Chang, J., Zhang, C., Fu, Z., Zang, X., Guan, L., Lu, J., Hui, Y., Leng, D., Niu, Y., Song, Y., and Gai, K. Twin: Two-stage interest network for lifelong user behavior modeling in ctr prediction at kuaishou, 2023.

Chen, Q., Zhao, H., Li, W., Huang, P., and Ou, W. Behavior sequence transformer for e-commerce recommendation in alibaba. In *Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data*, DLP-KDD '19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367837. doi: 10.1145/3326937.3341261. URL https://doi.org/10.1145/3326937.3341261.

Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20, 2020.

Cheng, H.-T., Koc, L., Harmsen, J., Shaked, T., Chandra, T., Aradhye, H., Anderson, G., Corrado, G., Chai, W., Ispir, M., Anil, R., Haque, Z., Hong, L., Jain, V., Liu, X., and Shah, H. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, DLRS 2016, pp. 7–10, 2016. ISBN 9781450347952.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *CoRR*, abs/1904.10509, 2019. URL http://arxiv.org/abs/1904.10509.

Covington, P., Adams, J., and Sargin, E. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, RecSys '16, pp. 191–198, 2016. ISBN 9781450340359.

Cui, Z., Ma, J., Zhou, C., Zhou, J., and Yang, H. M6-rec: Generative pretrained language models are open-ended recommender systems, 2022.

Dallmann, A., Zoller, D., and Hotho, A. A case study on sampling strategies for evaluating neural sequential item recommendation models. In *Proceedings of the 15th ACM Conference on Recommender Systems*, RecSys '21, pp. 505–514, 2021. ISBN 9781450384582.

Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

Devlin, J., Chang, M., Lee, K., and Toutanova, K. BERT: pre-training of deep bidirectional transformers for language understanding. In Burstein, J., Doran, C., and Solorio, T. (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pp. 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/n19-1423. URL https://doi.org/10.18653/v1/n19-1423.

Eksombatchai, C., Jindal, P., Liu, J. Z., Liu, Y., Sharma, R., Sugnet, C., Ulrich, M., and Leskovec, J. Pixie: A system for recommending 3+ billion items to 200+ million users in real-time. In *Proceedings of the 2018 World Wide Web Conference*, WWW '18, pp. 1775–1784, 2018. ISBN 9781450356398.

Elfwing, S., Uchibe, E., and Doya, K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *CoRR*, abs/1702.03118, 2017. URL http://arxiv.org/abs/1702.03118.

Gao, W., Fan, X., Wang, C., Sun, J., Jia, K., Xiao, W., Ding, R., Bin, X., Yang, H., and Liu, X. Learning an end-to-end structure for retrieval in large-scale recommendations. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*, CIKM '21, pp. 524–533, 2021. ISBN 9781450384469.

Gillenwater, J., Kulesza, A., Fox, E., and Taskar, B. Expectation-maximization for learning determinantal point processes. In *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2*, NIPS'14, pp. 3149–3157, Cambridge, MA, USA, 2014. MIT Press.

Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL https://openreview.net/forum?id=uYLFoz1vlAC.

Guo, H., Tang, R., Ye, Y., Li, Z., and He, X. Deepfm: A factorization-machine based neural network for ctr prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, IJCAI'17, pp. 1725–1731, 2017. ISBN 9780999241103.

Gupta, M. R., Bengio, S., and Weston, J. Training highly multiclass classifiers. *J. Mach. Learn. Res.*, 15(1):1461–1492, jan 2014. ISSN 1532-4435.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015.

He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., and Candela, J. Q. Practical lessons from predicting clicks on ads at facebook. In *ADKDD'14: Proceedings of the Eighth International Workshop on Data Mining for Online Advertising*, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450329996.

Hidasi, B., Karatzoglou, A., Baltrunas, L., and Tikk, D. Session-based recommendations with recurrent neural networks. In Bengio, Y. and LeCun, Y. (eds.), *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. URL http://arxiv.org/abs/1511.06939.

Hou, Y., Zhang, J., Lin, Z., Lu, H., Xie, R., McAuley, J., and Zhao, W. X. Large language models are zero-shot rankers for recommender systems. In *Advances in Information Retrieval - 46th European Conference on IR Research, ECIR 2024*, 2024.

Hua, W., Dai, Z., Liu, H., and Le, Q. V. Transformer quality in linear time. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvári, C., Niu, G., and Sabato, S. (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 9099–9117. PMLR, 2022. URL https://proceedings.mlr.press/v162/hua22a.html.

Huang, G., Sun, Y., Liu, Z., Sedra, D., and Weinberger, K. Deep networks with stochastic depth, 2016.

Jegou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, jan 2011. ISSN 0162-8828. doi: 10.1109/TPAMI.2010.57. URL https://doi.org/10.1109/TPAMI.2010.57.

Kang, W.-C. and McAuley, J. Self-attentive sequential recommendation. In *2018 International Conference on Data Mining (ICDM)*, pp. 197–206, 2018.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020. URL https://arxiv.org/abs/2001.08361.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are rnns: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.

Khudia, D., Huang, J., Basu, P., Deng, S., Liu, H., Park, J., and Smelyanskiy, M. Fbgemm: Enabling high-performance low-precision deep learning inference. *arXiv preprint arXiv:2101.05615*, 2021.

Klenitskiy, A. and Vasilev, A. Turning dross into gold loss: is bert4rec really better than sasrec? In *Proceedings of the 17th ACM Conference on Recommender Systems*, RecSys '23, pp. 1120–1125, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702419. doi: 10.1145/3604915.3610644. URL https://doi.org/10.1145/3604915.3610644.

Korthikanti, V., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoeybi, M., and Catanzaro, B. Reducing activation recomputation in large transformer models, 2022.

Li, C., Chang, E., Garcia-Molina, H., and Wiederhold, G. Clustering for approximate similarity search in high-dimensional spaces. *IEEE Transactions on Knowledge and Data Engineering*, 14(4):792–808, 2002.

Li, J., Wang, M., Li, J., Fu, J., Shen, X., Shang, J., and McAuley, J. Text is all you need: Learning language representations for sequential recommendation. In *KDD*, 2023.

Liu, Z., Zou, L., Zou, X., Wang, C., Zhang, B., Tang, D., Zhu, B., Zhu, Y., Wu, P., Wang, K., and Cheng, Y. Monolith: Real time recommendation system with collisionless embedding table, 2022.

Ma, J., Zhao, Z., Yi, X., Chen, J., Hong, L., and Chi, E. H. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. KDD '18, 2018.

Mudigere, D., Hao, Y., Huang, J., Jia, Z., Tulloch, A., Sridharan, S., Liu, X., Ozdal, M., Nie, J., Park, J., Luo, L., Yang, J. A., Gao, L., Ivchenko, D., Basant, A., Hu, Y., Yang, J., Ardestani, E. K., Wang, X., Komuravelli, R., Chu, C.-H., Yilmaz, S., Li, H., Qian, J., Feng, Z., Ma, Y., Yang, J., Wen, E., Li, H., Yang, L., Sun, C., Zhao, W., Melts, D., Dhulipala, K., Kishore, K., Graf, T., Eisenman, A., Matam, K. K., Gangidi, A., Chen, G. J., Krishnan, M., Nayak, A., Nair, K., Muthiah, B., khorashadi, M., Bhattacharya, P., Lapukhov, P., Naumov, M., Mathews, A., Qiao, L., Smelyanskiy, M., Jia, B., and Rao, V. Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ISCA '22, pp. 993–1011, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450386104. doi: 10.1145/3470496.3533727. URL https://doi.org/10.1145/3470496.3533727.

Peng, B., Quesnelle, J., Fan, H., and Shippole, E. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=wHBfxhZu1u.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference, 2022.

Press, O., Smith, N. A., and Lewis, M. Train short, test long: Attention with linear biases enables input length extrapolation. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL https://openreview.net/forum?id=R8sQPpGCv0.

Rabe, M. N. and Staats, C. Self-attention does not need $o(n^2)$ memory, 2021.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(1), jan 2020. ISSN 1532-4435.

Rendle, S. Factorization machines. In *2010 IEEE International Conference on Data Mining (ICDM)*, pp. 995–1000, 2010. doi: 10.1109/ICDM.2010.127.

Rendle, S., Krichene, W., Zhang, L., and Anderson, J. Neural collaborative filtering vs. matrix factorization revisited. In *Fourteenth ACM Conference on Recommender Systems (RecSys'20)*, pp. 240–248, 2020. ISBN 9781450375832.

Shazeer, N. Glu variants improve transformer, 2020.

Shin, K., Kwak, H., Kim, S. Y., Ramström, M. N., Jeong, J., Ha, J.-W., and Kim, K.-M. Scaling law for recommendation models: towards general-purpose user representations. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI'23/IAAI'23/EAAI'23. AAAI Press, 2023. ISBN 978-1-57735-880-0. doi: 10.1609/aaai.v37i4.25582. URL https://doi.org/10.1609/aaai.v37i4.25582.

Shrivastava, A. and Li, P. Asymmetric lsh (alsh) for sublinear time maximum inner product search (mips). In *Advances in Neural Information Processing Systems*, volume 27, 2014.

Sileo, D., Vossen, W., and Raymaekers, R. Zero-shot recommendation as language modeling. In Hagen, M., Verberne, S., Macdonald, C., Seifert, C., Balog, K., Nørvåg, K., and Setty, V. (eds.), *Advances in Information Retrieval - 44th European Conference on IR Research, ECIR 2022, Stavanger, Norway, April 10-14, 2022, Proceedings, Part II*, volume 13186 of *Lecture Notes in Computer Science*, pp. 223–230. Springer, 2022. doi: 10.1007/978-3-030-99739-7_26. URL https://doi.org/10.1007/978-3-030-99739-7_26.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding, 2023.

Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., and Jiang, P. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, CIKM '19, pp. 1441–1450, 2019. ISBN 9781450369763.

Tang, H., Liu, J., Zhao, M., and Gong, X. Progressive layered extraction (ple): A novel multi-task learning (mtl) model for personalized recommendations. In *Proceedings of the 14th ACM Conference on Recommender Systems*, RecSys '20, pp. 269–278, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450375832. doi: 10.1145/3383313.3412236. URL https://doi.org/10.1145/3383313.3412236.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models, 2023a.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models, 2023b.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, pp. 6000–6010, 2017. ISBN 9781510860964.

Wang, R., Shivanna, R., Cheng, D., Jain, S., Lin, D., Hong, L., and Chi, E. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the Web Conference 2021*, WWW '21, pp. 1785–1797, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383127. doi: 10.1145/3442381.3450078. URL https://doi.org/10.1145/3442381.3450078.

Wang, Z., Zhao, L., Jiang, B., Zhou, G., Zhu, X., and Gai, K. Cold: Towards the next generation of pre-ranking system, 2020.

Xia, X., Eksombatchai, P., Pancha, N., Badani, D. D., Wang, P.-W., Gu, N., Joshi, S. V., Farahpour, N., Zhang, Z., and Zhai, A. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, pp. 5249–5259, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701030. doi: 10.1145/3580305.3599918. URL https://doi.org/10.1145/3580305.3599918.

Xiao, J., Ye, H., He, X., Zhang, H., Wu, F., and Chua, T.-S. Attentional factorization machines: Learning the weight of feature interactions via attention networks. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, IJCAI'17, pp. 3119–3125. AAAI Press, 2017. ISBN 9780999241103.

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., and Liu, T.-Y. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.

Yang, J., Yi, X., Zhiyuan Cheng, D., Hong, L., Li, Y., Xiaoming Wang, S., Xu, T., and Chi, E. H. Mixed negative sampling for learning two-tower neural networks in recommendations. In *Companion Proceedings of the Web Conference 2020*, WWW '20, pp. 441–447, 2020. ISBN 9781450370240.

Zhai, J., Lou, Y., and Gehrke, J. Atlas: A probabilistic algorithm for high dimensional similarity search. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data*, SIGMOD '11, pp. 997–1008, 2011. ISBN 9781450306614.

Zhai, J., Gong, Z., Wang, Y., Sun, X., Yan, Z., Li, F., and Liu, X. Revisiting neural retrieval on accelerators. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, pp. 5520–5531, New York, NY, USA, 2023a. Association for Computing Machinery. ISBN 9798400701030. doi: 10.1145/3580305.3599897. URL https://doi.org/10.1145/3580305.3599897.

Zhai, Y., Jiang, C., Wang, L., Jia, X., Zhang, S., Chen, Z., Liu, X., and Zhu, Y. Bytetransformer: A high-performance transformer boosted for variable-length inputs. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 344–355, Los Alamitos, CA, USA, may 2023b. IEEE Computer Society. doi: 10.1109/IPDPS54959.2023.00042. URL https://doi.ieeecomputersociety.org/10.1109/IPDPS54959.2023.00042.

Zhang, B., Luo, L., Liu, X., Li, J., Chen, Z., Zhang, W., Wei, X., Hao, Y., Tsang, M., Wang, W., Liu, Y., Li, H., Badr, Y., Park, J., Yang, J., Mudigere, D., and Wen, E. Dhen:

A deep and hierarchical ensemble network for large-scale click-through rate prediction, 2022.

Zhao, X., Xia, L., Zhang, L., Ding, Z., Yin, D., and Tang, J. Deep reinforcement learning for page-wise recommendations. In *Proceedings of the 12th ACM Conference on Recommender Systems*, RecSys '18, pp. 95–103, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450359016. doi: 10.1145/3240323.3240374. URL https://doi.org/10.1145/3240323.3240374.

Zhao, Z., Yang, Y., Wang, W., Liu, C., Shi, Y., Hu, W., Zhang, H., and Yang, S. Breaking the curse of quality saturation with user-centric ranking, 2023.

Zhou, G., Zhu, X., Song, C., Fan, Y., Zhu, H., Ma, X., Yan, Y., Jin, J., Li, H., and Gai, K. Deep interest network for click-through rate prediction. KDD '18, 2018.

Zhou, K., Wang, H., Zhao, W. X., Zhu, Y., Wang, S., Zhang, F., Wang, Z., and Wen, J.-R. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, CIKM '20, pp. 1893–1902, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450368599. doi: 10.1145/3340531.3411954. URL https://doi.org/10.1145/3340531.3411954.

Zhuo, J., Xu, Z., Dai, W., Zhu, H., Li, H., Xu, J., and Gai, K. Learning optimal tree models under beam search. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.

A. 符号说明

我们在 Table 8 和 Table 9 中汇总了本文使用的主要符号。

B. 生成式推荐器：背景与问题表述

鉴于经典深度学习推荐模型（DLRM）(Mudigere et al., 2022) 自 YouTube DNN 时代 (Covington et al., 2016) 起便广为流行，并且普遍用于每一个大型在线内容及电子商务平台 (Cheng et al., 2016; Zhou et al., 2018; Wang et al., 2021; Chang et al., 2023; Xia et al., 2023; Zhai et al., 2023a)，许多读者可能对它更为熟悉。DLRM 在异构特征空间上使用各种神经网络，包括特征交互模块 (Guo et al., 2017; Xiao et al., 2017; Wang et al., 2021)、序列池化或目标感知成对注意力模块 (Hidasi et al., 2016; Zhou et al., 2018; Chang et al., 2023)，以及先进的多专家多任务模块 (Ma et al., 2018; Tang et al., 2020)。因此，我们在 Section 2 和 Section 3 中明确对比了生成式推荐器（GR）与经典 DLRM，以概述 GR。本节从经典序列推荐文献出发，为读者提供另一种视角。

B.1. 背景：学术界与工业界的序列推荐

B.1.1. 学术研究（传统序列推荐器设置）

循环神经网络（RNN）首次通过 GRU4Rec (Hidasi et al., 2016) 应用于推荐场景。Hidasi et al. (2016) 考虑了门控循环单元（GRU），并将其应用于 RecSys Challenge 2015 [2] 和 VIDEO（一个专有数据集）这两个数据集。在两种情况下，输入序列中都*只保留正向事件（被点击的电商物品，或用户观看时间达到一定阈值的视频）*。我们进一步观察到，在由召回和排序阶段组成的经典工业级两阶段推荐系统 (Covington et al., 2016) 中，Hidasi et al. (2016) 所解决的任务主要对应召回任务。

Transformer、序列转导架构及其变体。此后序列转导架构，尤其是 Transformer (Vaswani et al., 2017) 的进步，也推动了推荐系统的类似发展。SASRec (Kang & McAuley, 2018) 首次在自回归设置中应用 Transformer。它将评论或评分的*出现*视为*正反馈*，从而与 GRU4Rec 类似，将 Amazon Reviews [3] 和 MovieLens [4] 等经典数据集*转化为正向物品序列*。该方法采用二元交叉熵损失，将下一个“正向”物品定义为正目标（注意，这本质上仅指存在评论或评分），而负目标从物品集合 $\mathbb{X} = \mathbb{X}_c$ 中随机采样。

此后的大多数研究都建立在上述 GRU4Rec (Hidasi et al., 2016) 和 SASRec (Kang & McAuley, 2018) 的类似设置之上，例如 BERT4Rec (Sun et al., 2019) 应用 BERT (Devlin et al., 2019) 的双向编码器设置，S3Rec (Zhou et al., 2020) 引入显式预训练阶段，等等。

B.1.2. 作为深度学习推荐模型（DLRM）组成部分的工业应用

序列方法，包括序列编码器和成对注意力模块，因能够作为 DLRM 的一部分增强用户表示，已广泛应用于工业场景。DLRM 通常使用相对较短的序列，例如 BST (Chen et al., 2019) 使用 20，DIN (Zhou et al., 2018) 使用 1,000，TransAct (Xia et al., 2023) 使用 100。我们注意到，这些长度比本文的 8,192（Section 4.3）小 1–3 个数量级。

尽管序列长度较短，大多数 DLRM 仍能够成功捕获用户的长期偏好。这主要归因于两点。首先，现代 DLRM 普遍使用预先计算的用户画像/嵌入 (Xia et al., 2023) 或外部向量存储 (Chang et al., 2023)，两者都有效延长了历史回溯窗口。其次，模型通常采用大量上下文特征、用户侧特征和物品侧特征 (Zhou et al., 2018; Chen et al., 2019; Chang et al., 2023; Xia et al., 2023)，并通过 FM (Xiao et al., 2017; Guo et al., 2017)、DCN (Wang et al., 2021)、MoE 等异构网络变换表示并组合输出。

与 Appendix B.1.1 中讨论的序列设置不同，所有主要工业研究都在（用户/请求，候选物品）对上定义损失。在排序设置下，通常采用多任务二元交叉熵损失。在召回设置下，双塔结构 (Covington et al., 2016) 仍是主流方法。近期研究探索将下一个待推荐物品表示为（子）token 序列上的概率分布，例如 OTM (Zhuo et al., 2020) 和 DR (Gao et al., 2021)（注意，其他一些近期工作也将同样的设置称为“生成式召回”）。这些方法通常通过束搜索从子 token 解码出物品。随着 GPU、定制 ASIC 和 TPU 等现代加速器的普及，混合 logits (Zhai et al., 2023a) 等先进的可学习相似度函数也已被提出并部署，作为双塔结构和束搜索的替代方案。

[2] http://2015.recsyschallenge.com/
[3] https://jmcauley.ucsd.edu/data/amazon/
[4] https://grouplens.org/datasets/movielens/1m/, https://grouplens.org/datasets/movielens/20m/

从问题表述角度，我们将上述所有工作都视为 DLRM (Mudigere et al., 2022) 的一部分，因为其模型架构、所用特征和损失均与 Appendix B.1.1 中讨论的学术序列推荐研究有显著区别。还值得指出，在本文之前，工业界尚无完全序列化排序设置的成功应用，尤其没有在十亿日活跃用户（DAU）规模上的成功应用。

B.2. 问题表述：生成式推荐器（GR）中作为序列转导任务的排序与召回

接下来，我们讨论传统序列推荐器设置和 DLRM 设置的三个局限，以及生成式推荐器（GR）如何从问题表述角度解决这些局限。

忽略用户交互物品之外的特征。以往的序列化表述只考虑用户显式交互过的内容（物品）(Hidasi et al., 2016; Kang & McAuley, 2018; Sun et al., 2019; Zhou et al., 2020)，而在 GR 出现之前，工业级推荐系统就已使用海量特征进行训练，以增强用户和内容表示 (Covington et al., 2016; Cheng et al., 2016; Zhou et al., 2018; Chen et al., 2019; Chang et al., 2023; Xia et al., 2023; Zhai et al., 2023a)。GR 通过以下方式解决这一局限：a）压缩其他类别特征，并将其与主时间序列合并；b）采用目标感知的表述，通过交叉注意力交互捕获数值特征，如 Section 2.1 和 Figure 2 所述。我们通过实验验证了这一点：忽略此类特征的传统“仅交互”表述会显著降低模型质量。实验结果见 Table 7 和 Table 6 中标记为“GR (interactions only)”的行；仅使用交互历史使召回的 hit rate@100 降低 1.3%，排序的 NE 下降 2.6%（注意，如 Sections 4.1.2 and 4.3.1 所述，NE 变化 0.1% 已具有显著影响）。

在与目标无关的设置下计算用户表示。第二个问题是，大多数传统序列推荐器，包括 GRU4Rec (Hidasi et al., 2016)、SASRec (Kang & McAuley, 2018)、BERT4Rec (Sun et al., 2019)、S3Rec (Zhou et al., 2020) 等，采用与目标无关的表述：对于目标物品 Φ_i，以 $\Phi_0, \Phi_1, \ldots, \Phi_{i-1}$ 作为编码器输入来计算用户表示，再用该表示进行预测。相比之下，工业场景中大多数主要 DLRM 方法都将所用序列模块表述为目标感知形式，能够将“目标”（排序候选项）信息融入用户表示。这些方法包括 DIN (Zhou et al., 2018)（阿里巴巴）、BST (Chen et al., 2019)（阿里巴巴）、TWIN (Chang et al., 2023)（快手）以及 TransAct (Xia et al., 2023)（Pinterest）。

生成式推荐器（GR）通过*交错排列*内容和行为序列（Section 2.2），兼顾了两种方法的优势，使目标感知注意力能够用于因果自回归设置。我们在 Table 10 中对既有工作与本文进行了分类和比较 [5]。

判别式表述使此前的序列推荐研究局限于逐点设置。最后，传统序列推荐器在设计上是判别式的。包括 GRU4Rec 和 SASRec 等开创性工作在内的既有序列推荐文献，建模的是 $p(\Phi_i|\Phi_0, a_0, \ldots, \Phi_{i-1}, a_{i-1})$，即给定用户当前状态时，下一个待推荐物品的条件分布。另一方面，我们观察到，标准推荐系统包含两个概率过程，即*推荐系统向用户推荐内容 Φ_i（例如某张照片或某段视频）的过程，以及用户对所推荐内容 Φ_i 以某种行为 a_i 作出回应的过程*（该行为可以是点赞、完播、跳过等的组合）。

如 Section 2.2 所述，生成式方法需要对所推荐内容与用户行为序列的联合分布建模，即 $p(\Phi_0, a_0, \Phi_1, a_1, \ldots, \Phi_{n_c-1}, a_{n_c-1})$。我们提出的*生成式推荐器*能够对这类分布建模，如 Table 11（Figure 8）所示。注意，下一个行为 token（a_i）预测任务，恰好就是 Table 1 中讨论的 GR 排序设置；而下一个内容（Φ_i）预测任务则类似于适配到交错序列的召回设置，但更改了目标以学习输入数据分布。

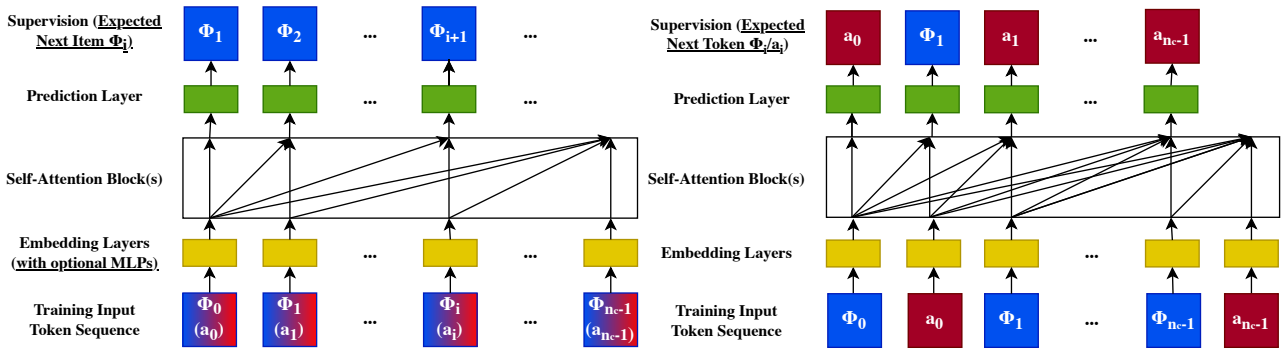


Figure 8. 传统序列推荐器（左）与生成式推荐器（右）的比较。为便于比较，我们展示因果自回归设置下的序列推荐器，以及不含上下文特征的 GR。左侧中，行为类型 a_i 在进入自注意力块之前，要么被忽略，要么通过 MLP 与物品信息 Φ_i 组合。

[5]由于记录的数据量巨大，大多数大规模工业推荐器需要在流式/单遍设置下训练。

[6]BERT4Rec 使用多遍训练，并混合完形填空损失和逐点（最后一个物品）监督损失；S3Rec 采用多遍训练，将预训练和微调作为两个独立阶段。

重要的是，这一表述不仅能够恰当地建模数据分布，还支持通过束搜索等方法直接采样出向用户推荐的物品序列。我们推测，与传统列表级设置（例如 DPP (Gillenwater et al., 2014) 和 RL (Zhao et al., 2018)）相比，这将带来更优的方法；此类系统的完整表述和评估（在 Section 6 中作了简要讨论）留作未来工作。

C. 评估：合成数据

如 Section 3.1 所述，标准 softmax 注意力的归一化因子使其难以捕获用户偏好的强度，而这一点对于用户表示学习很重要。推荐场景中尤其如此，因为系统除了需要预测物品的相对顺序，还可能需要预测互动强度（例如未来在某个主题上的正向行为次数）。

为理解这一行为，我们构造了遵循狄利克雷过程的合成数据，该过程在动态词表上生成流式数据。狄利克雷过程能够捕获用户互动历史中的“富者愈富”现象。合成实验设置如下：

- 我们将 20,000 个物品 ID 中的每一个随机分配给 100 个类别中的唯一一个类别。
- 我们生成 1,000,000 条记录，每条长度为 128，前 90% 用于训练，最后 10% 用于测试。为模拟流式训练设置，我们最初只开放前 40% 的物品 ID，其余 ID 以相等间隔逐步开放；也就是说，在第 500,000 条记录处，可采样的最大 ID 为 $(40\% + 60\% * 0.5) * 20,000 = 14,000$。
- 对于每条记录，我们从 100 个类别中随机选择最多 5 个类别，并在这 5 个类别上随机采样先验 H_c。随后，按照可能类别上的狄利克雷过程，依次为每个位置采样类别，过程如下：
  - 对于 $n > 1$：
    * 以概率 $\alpha/(\alpha + n - 1)$，抽取类别 c，其来源为 H_c。
    * 以概率 $n_c/(\alpha + n - 1)$，抽取类别 c，其中 n_c 是此前属于类别 c 的物品数量。
    * 在流式约束下，随机采样一个分配给类别 c 的物品。

  其中 α 从 $(1.0, 500.0)$ 中均匀随机采样。

结果见表 2。由于该数据集没有时间戳，我们始终移除 HSTU 的 $\text{rab}^{p,t}$。我们观察到，相较标准 Transformer，HSTU 将 Hit Rate@10 提升了超过 100%。重要的是，将 HSTU 的逐点注意力机制替换为 softmax（“HSTU w/ Softmax”）同样会使命中率显著下降，这验证了类似逐点注意力的聚合机制的重要性。

D. 评估：传统序列推荐器设置

我们在 Section 4.1.1 中的评估，主要将 HSTU 与采用最新训练方案的先进 Transformer 基线 SASRec 进行比较。本节进一步考虑另外两种方法。

循环神经网络（RNN）。我们考虑序列推荐领域的经典工作 GRU4Rec (Hidasi et al., 2016)，以帮助读者了解：在充分纳入最新建模与训练改进之后，包括 Transformer 和 HSTU 在内的自注意力模型与传统 RNN 相比表现如何。

自监督序列方法。我们考虑最流行的工作 BERT4Rec (Sun et al., 2019)，以了解双向自监督（BERT4Rec 通过完形填空目标使用它）与 SASRec、HSTU 等单向因果自回归设置相比的表现。

结果见 Table 12。我们复用了 Klenitskiy & Vasilev (2023) 报告的 BERT4Rec 和 GRU4Rec 在 ML-1M、ML-20M 上的结果。由于使用采样 softmax 损失，我们固定所用负样本数量（ML-1M、ML-20M 为 128，Amazon Books 为 512），以确保各方法之间的公平比较。

结果证实，在采用采样 softmax 损失时，SASRec 仍是传统序列推荐设置中最具竞争力的方法之一 (Zhai et al., 2023a; Klenitskiy & Vasilev, 2023)；而 HSTU 显著优于本次评估的 Transformer、RNN 和自监督双向 Transformer。

E. 评估：传统 DLRM 基线

Section 4 所用 DLRM 基线配置凝聚了数百名研究者和工程师多年的持续迭代，近似对应一个拥有数十亿日活跃用户的大型互联网平台在部署 HSTU/GR 之前的生产配置。下面概述所用模型。

排序设置。如 (Mudigere et al., 2022) 所述，基线排序模型使用约一千个稠密特征和五十个稀疏特征。我们纳入了多种建模技术，包括混合专家 (Ma et al., 2018)、Deep & Cross Network 的变体 (Wang et al., 2021)、包括目标感知成对注意力在内的各种序列推荐模块（工业场景中常用的一种变体见 (Zhou et al., 2018)），以及特殊交互层上的

残差连接 (He et al., 2015; Zhang et al., 2022)。对于缩放定律部分（Section 4.3.1）的低 FLOPs 区域，我们简化了部分计算成本较高的模块，或用 DCN 等其他先进变体加以替换，以达到所需 FLOPs。

由于保密原因，我们无法披露精确配置；但据我们所知，在充分纳入近期研究后，我们的基线代表了已知最佳 DLRM 方法之一。为验证这一说法并方便读者理解，我们报告了一种使用相同特征、但仅采用 DIN (Zhou et al., 2018)、DCN (Wang et al., 2021) 和 MMoE (Ma et al., 2018) 等主要已发表成果的典型配置（“DLRM (DIN+DCN)”），结果见 Table 7，其组合架构如 Figure 9 所示。该配置明显落后于我们的生产 DLRM：主要 E-Task 的 NE 差 0.71%，主要 C-Task 的 NE 差 0.57%（其中 0.1% 的 NE 变化已具有显著影响）。

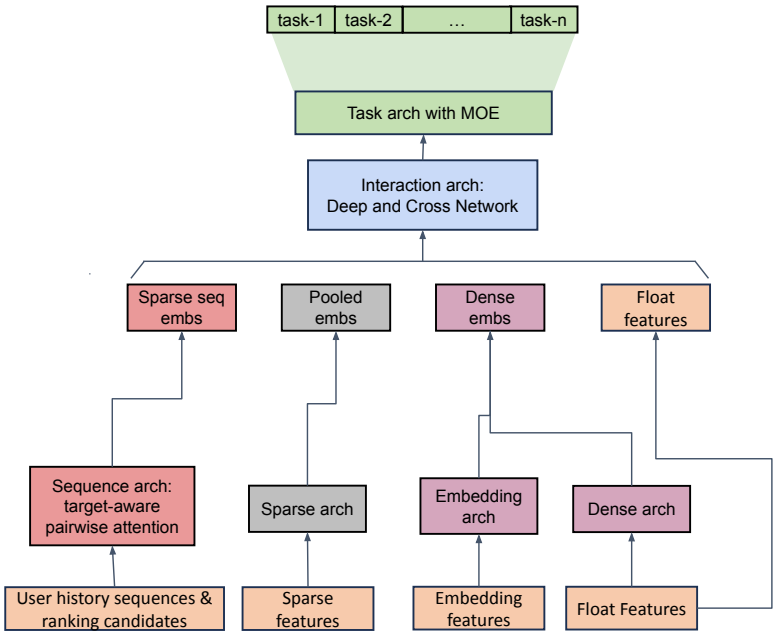


Figure 9. 基线 DLRM 排序模型（Table 7 中的“DLRM (DIN+DCN)”）的总体架构，采用 DIN (Zhou et al., 2018)、DCN (Wang et al., 2021) 和 MMoE (Ma et al., 2018) 等主要已发表工作。

召回设置。基线召回模型采用标准双塔神经召回设置 (Covington et al., 2016)，混合使用批内采样和批外采样。输入特征集合同时包含高基数稀疏特征（例如物品 ID、用户 ID）与低基数稀疏特征（例如语言、主题、兴趣实体）。模型使用带残差连接 (He et al., 2015) 的多层前馈网络，将输入特征压缩为用户嵌入和物品嵌入。

特征与序列长度。两个 DLRM 基线使用的特征，包括各种序列编码器/成对注意力模块所使用的主要用户交互历史，都是所有 GR 候选模型所用特征的*严格超集*。这一点适用于本文开展的全部研究，包括缩放研究（Section 4.3.1）。

F. 随机长度

F.1. 子序列选择

在 Equation (4) 中，我们从完整用户历史中选择长度为 L 的子序列，以提高稀疏性。实证结果表明，精心设计子序列选择技术可以改善模型质量。我们计算指标 $f_i = t_n - t_i$，它对应用户与物品 x_i 交互以来经过的时间。我们使用以下子序列选择方法开展离线实验：

- **贪心选择**——选择 L 个具有最小 f_i 值的物品，选择范围为 S
- **随机选择**——随机选择 L 个物品，选择范围为 S
- **特征加权选择**——选择 L 个物品，从 S 中按加权分布 $1 - f_{n,i}/(\sum_{j=1}^{L} f_{j,i})$ 采样

在离线实验中，特征加权的子序列选择方法取得了最佳模型质量，如 Table 13 所示。

F.2. 随机长度对序列稀疏率的影响

在 Table 3 中，我们针对使用 30 天用户互动历史的代表性工业级配置，展示了随机长度对序列稀疏率的影响。序列稀疏率定义为 1 减去所有样本的平均序列长度与最大序列长度之比。为更好地刻画稀疏注意力的计算成本，我们还定义了 *s2*，其值为 1 减去注意力矩阵的稀疏率。作为参考，我们在 Table 14 和 Table 15 中分别给出使用 60 天和 90 天用户互动历史的结果。

F.3. 与序列长度外推技术的比较

我们开展了额外研究，以验证*随机长度*与语言建模中现有序列长度外推技术相比具有竞争力。许多现有方法通过修改 RoPE (Su et al., 2023) 来实现序列长度外推。为与现有方法比较，我们训练了一种不使用相对注意力偏置、而使用旋转嵌入的 HSTU 变体（HSTU-RoPE）。

我们在 HSTU-RoPE 上评估以下序列长度外推方法：

- **零样本**——应用 NTK-Aware RoPE (Peng et al., 2024) 后，不进行微调，直接评估模型；
- **微调**——应用 NTK-by-parts (Peng et al., 2024) 后，对模型微调 1000 步。

我们在 HSTU（包含相对注意力偏置，不含旋转嵌入）上评估以下序列长度外推方法：

- **零样本**——根据训练时最大序列长度截断相对位置偏置的取值范围，直接评估模型 (Raffel et al., 2020; Press et al., 2022)；
- **微调**——根据训练时最大序列长度截断相对位置偏置的取值范围，先对模型微调 1000 步，再进行评估。

在 Table 16 中，我们报告训练时引入数据稀疏性的模型（随机长度、零样本、微调）与使用完整数据训练的模型之间的 NE 差异。对于零样本和微调技术，我们将稀疏率定义为训练时平均序列长度除以评估时最大序列长度。所有零样本和微调模型均在序列长度为 1024 的数据上训练，并在长度为 2048 和 4096 的数据上评估。为给这些技术找到合适的*随机长度*基线，我们选择能产生相同数据稀疏率指标的*随机长度*配置。

我们认为，采用零样本和微调进行序列长度外推的方法，不太适合处理高基数 ID 的推荐场景。实证结果显示，*随机长度*显著优于微调和零样本方法。我们认为，这可能是由于词表规模很大。零样本和微调方法无法为较早的 ID 学到良好表示，这可能削弱其充分利用较长序列所含信息的能力。

G. 稀疏分组 GEMM 与融合的相对注意力偏置

我们补充介绍 Section 3.2 中提出的高效 HSTU 注意力内核。该方法基于 Memory-efficient Attention (Rabe & Staats, 2021) 和 FlashAttention (Dao et al., 2022)，是一种内存高效的自注意力机制：它将输入分块，并避免为反向传播物化规模巨大的 $h \times N \times N$ 中间注意力张量。利用输入序列的稀疏性，我们可以将注意力计算重新表述为一组形状各异、连续执行的 GEMM。我们实现了高效 GPU 内核来加速这一计算。由于内存访问，相对注意力偏置的构造同样是一个瓶颈。为解决这一问题，我们将相对偏置构造与分组 GEMM 融合进单个 GPU 内核，并在反向传播中利用 GPU 的高速共享内存累积梯度。虽然该算法需要在反向传播中重新计算注意力和相对偏置，但其速度显著高于 Transformer 中使用的标准方法，内存占用也更低。

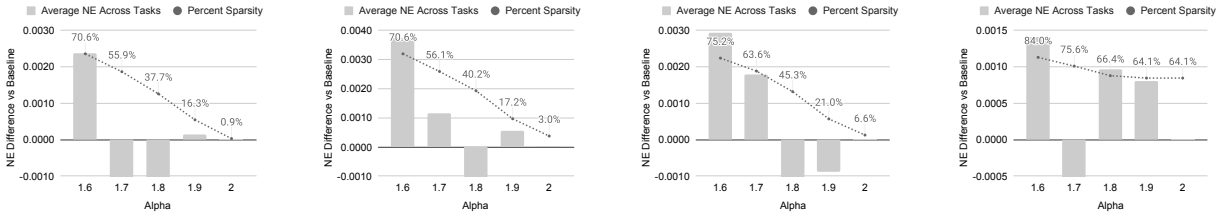
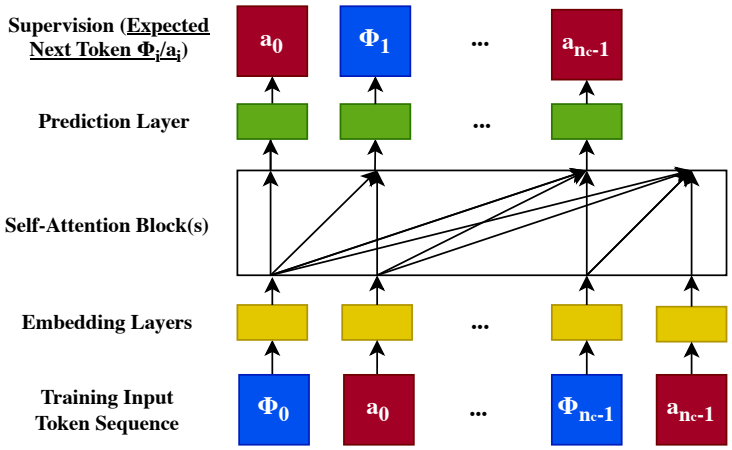


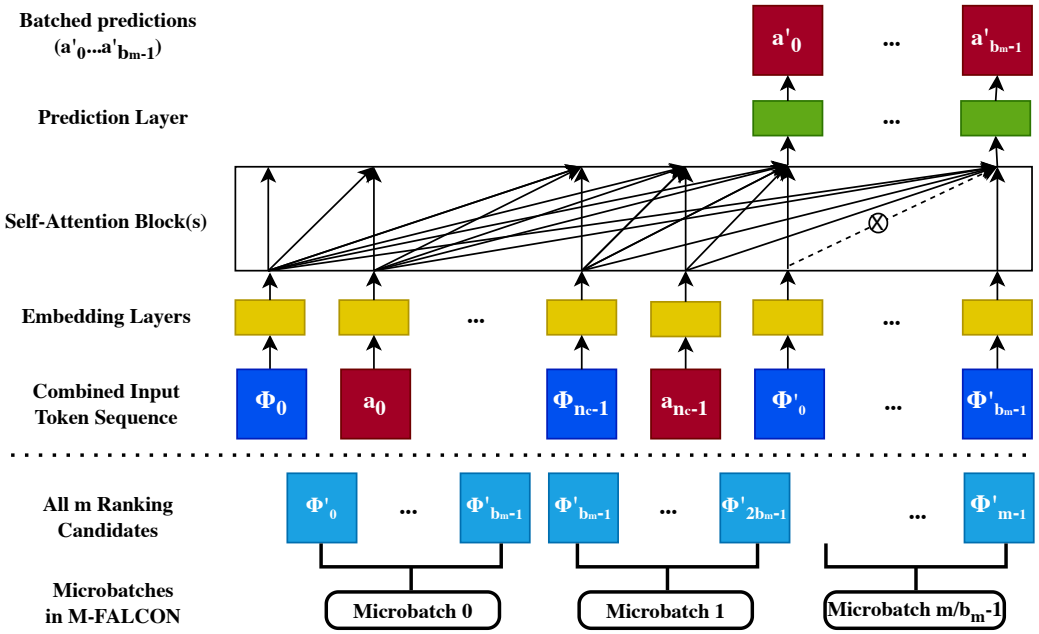
Figure 10. 随机长度（SL）对排序模型指标的影响。从左到右：$n = [1024, 2048, 4096, 8192]$（$n$ 是应用 Section 2.2 所述交错算法后的长度，该算法支持在因果掩码设置下使用目标感知交叉注意力）。

H. 利用可缓存运算的微批次快速注意力（M-FALCON）

本节详细介绍 Section 3.4 中讨论的 M-FALCON 算法。我们在 Algorithm 1 中给出 M-FALCON 的伪代码。M-FALCON 引入了三个关键思想。



(a) 在因果自回归设置下训练 GR 的排序模型（含 $n = 2n_c$ 个 token）。



(b) 使用 M-FALCON 算法对 GR 的排序模型执行推理。

Figure 11. M-FALCON 算法示意图。上：在 GR 的目标感知表述下进行模型训练。下：对 m 个候选项 $\Phi'_0, \ldots, \Phi'_{m-1}$ 进行模型推理，将它们分为 $\lceil m/b_m \rceil$ 个微批次；虚线以上展示了第一个微批次 $\Phi'_0, \ldots, \Phi'_{b_m-1}$ 的模型推理（总计 $2n_c + b_m$ 个 token，其中已计入 $\Phi_0, a_0, \ldots, \Phi_{n_c-1}, a_{n_c-1}$）。注意，自注意力算法经过修改，使 Φ'_i 无法关注 Φ'_j，条件为 $i \neq j$；图中以“×”突出表示这一点。

批量推理可以应用于因果自回归设置。如 Section 2.2 所述，GR 的排序任务采用目标感知表述。通常认为，在目标感知设置下，每次需要对一个物品执行推理，其成本为 $O(mn^2d)$，其中候选项数为 m，序列长度为 n。这里我们表明，这并非最优方案；即使使用普通 Transformer，也可以修改自注意力中的注意力掩码，对这些操作进行批处理（“批量推理”），将成本降至 $O((n+m)^2d) = O(n^2d)$。

示意见 Figure 11。其中，Figure 11（a）和（b）都涉及因果自回归设置的注意力掩码矩阵。关键区别在于，Figure 11（a）使用大小为 $2n_c$ 的标准下三角矩阵进行因果训练，而 Figure 11（b）修改了大小为 $2n_c + b_m$ 的下三角矩阵：对于 (i, j) 位置，当 $i, j \geq 2n_c, i \neq j$ 时，将其设为 False 或 $-\infty$，以防止目标位置 $\Phi'_0, \ldots, \Phi'_{b_m-1}$ 彼此关注。容易

看出，这样一来，自注意力块针对 Φ'_i 的输出 a'_i 只依赖于 $\Phi_0, a_0, \ldots, \Phi_{n_c-1}, a_{n_c-1}$，而不依赖于 Φ'_j（$i \neq j$）。换言之，使用修改后的注意力掩码对 $(2n_c + b_m)$ 个 token 执行一次前向处理，最后 b_m 个 token 得到的结果，与执行 b_m 次彼此独立、每次处理 $(2n_c + 1)$ 个 token 的前向处理相同；在后一种方式的每次处理中，Φ'_i 被放在第 $2n_c$ 个位置（从 0 开始计数），并在第 i 次前向处理中使用标准因果注意力掩码。

微批处理将批量推理扩展到大规模候选集合。排序阶段可能需要处理多达数以万计的候选项 (Wang et al., 2020)。我们可以将全部 m 个候选项划分为 $\lceil m/b_m \rceil$ 个大小为 b_m 的微批次，使 $O(b_m) = O(n)$，从而在大多数实际推荐设置中，即使候选项多达数以万计，仍保持前文讨论的 $O((n+m)^2 d) = O(n^2 d)$ 运行时间。

编码器层面的缓存使请求内及跨请求的计算共享成为可能。最后，KV 缓存 (Pope et al., 2022) 既可以用于请求内，也可以跨请求使用。例如，对于本文提出的 HSTU 模型（Section 3），$K(X)$ 和 $V(X)$ 可以在请求内和/或跨请求的微批次之间完全缓存。对于使用缓存的前向处理，我们只需计算 $U(X)$、$Q(X)$、$K(X)$ 和 $V(X)$，且仅针对最后 b_m 个 token；同时复用缓存的 $K(X)$ 和 $V(X)$，它们对应包含 n 个 token 的序列化用户历史。同样，$f_2(\text{Norm}(A(X)V(X)) \odot U(X))$ 也只需为 b_m 个候选项重新计算。这将使用缓存的前向处理的计算复杂度降至 $O(b_m d^2 + b_m n d)$，相较 $O((n+b_m)d^2 + (n+b_m)^2 d)$ 有显著改善；即使 $b_m = n$，也能带来 2–4 倍提升。

Algorithm 1 M-FALCON 算法。

Input: Merged token series $x_0, x_1, \ldots, x_{n-1}$ (can be e.g., $(\Phi_0, a_0, \ldots, \Phi_{n_c-1}, a_{n_c-1})$ where $n = 2n_c$); m ranking candidates $\Phi'_0, \ldots, \Phi'_{m-1}$; a b-layer h-heads self-attention model trained in causal autoregressive settings (e.g., HSTU or Transformers) $f(X, cacheStates, attnMask) \to (X', updatedCacheStates)$ where $X, X' \in \mathbb{R}^{N \times d}$, $attnMask \in \mathbb{R}^{N \times N}$, and $cachedStates, updatedCacheStates \in \mathbb{R}^{b \times h \times N \times d_{qk}} \times \mathbb{R}^{b \times h \times N \times d_{qk}}$ (due to caching $K(X)$s and $V(X)$s across b layers); microbatch size b_m, where we assume m is a multiple of b_m for simplicity.
Output: Predictions for all m ranking candidates, $(a'_0, \ldots, a'_{m-1})$.
$numMicrobatches = (m + b_m - 1)//b_m$
$attnMask = L_{n+b_m}$ {L_{n+b_m} represents a lower triangular matrix. Lower triangular entries are 0s, the rest are $-\infty$.}
$attnMask[i, j] = -\infty$ for $i, j \geq n, i \neq j$ {This prevents the last b_m entries from attending to each other.}
$(a'_0, a'_1, \ldots, a'_{b_m-1}), kvCache \leftarrow f(embLayer((x_0, x_1, \ldots, x_{n-1}, \Phi'_0, \ldots, \Phi'_{b_m-1})), \varnothing, attnMask)$
$predictions = (a'_0, a'_1, \ldots, a'_{b_m-1})$
$i = 1$
while $i < numMicrobatches$ **do**
  $(a'_{b_m i}, a'_{b_m i+1}, a'_{b_m(i+1)-1}), _ \leftarrow f(embLayer((x_0, x_1, \ldots, x_{n-1}, \Phi'_{b_m i}, \ldots, \Phi'_{b_m(i+1)-1})), kvCache, attnMask)$
  $predictions \leftarrow predictions + (a'_{b_m i}, a'_{b_m i+1}, \ldots, a'_{b_m(i+1)-1})$
  $i \leftarrow i + 1$
end while
return $predictions$

为便于理解，Algorithm 1 在 Figure 11 中作了图示。我们指出，M-FALCON 不仅适用于 HSTU 和 GR，也可作为推理优化算法，广泛应用于其他基于自注意力架构的目标感知因果自回归设置。

H.1. 推理吞吐量评估：使用 M-FALCON 的生成式推荐器（GR）与 DLRM

如 Section 3.4 所述，M-FALCON 并行处理 b_m 个候选项，从而在推理时将计算成本摊销到全部 m 个候选项上。为理解这一设计，我们在相同硬件配置下比较 GR 与 DLRM 的吞吐量，即每秒打分的候选项数量（QPS）。

如 Figure 12 和 Figure 13 所示，GR 的吞吐量随排序阶段候选项数量（m）以次线性方式扩展，直至某个范围——本案例中为 $m = 2048$——这是因为*批量推理*实现了成本摊销。这证实了批量推理在因果自回归设置中的关键作用。由于注意力复杂度按 $O((n+b_m)^2)$ 扩展，仅使用多个微批次本身就能提高吞吐量。缓存在微批处理基础上进一步消除了冗余的线性计算和注意力计算。如 Figure 13 所示，两者结合，相较采用单个微批次、$b_m = m = 1024$ 的基线，额外带来了最高 1.99 倍加速。总体而言，借助高效的 HSTU 编码器设计和 M-FALCON，基于 HSTU 的生成式推荐器在大规模生产配置中的吞吐量*优于* DLRM，最高达到其 2.99 倍，尽管 GR 的 *FLOPs 复杂度为后者的 285 倍*。

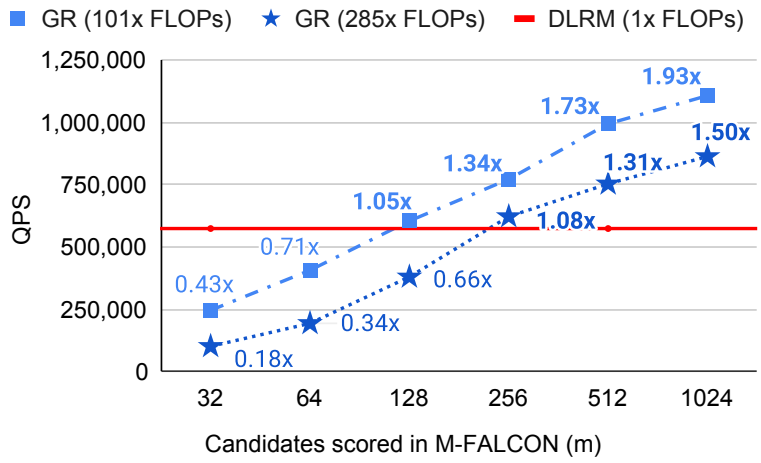


Figure 12. 端到端推理吞吐量：大规模工业场景下的 DLRM 与 GR（使用 M-FALCON）。注意，本图与 Figure 6 相同，在此重复给出以方便阅读。

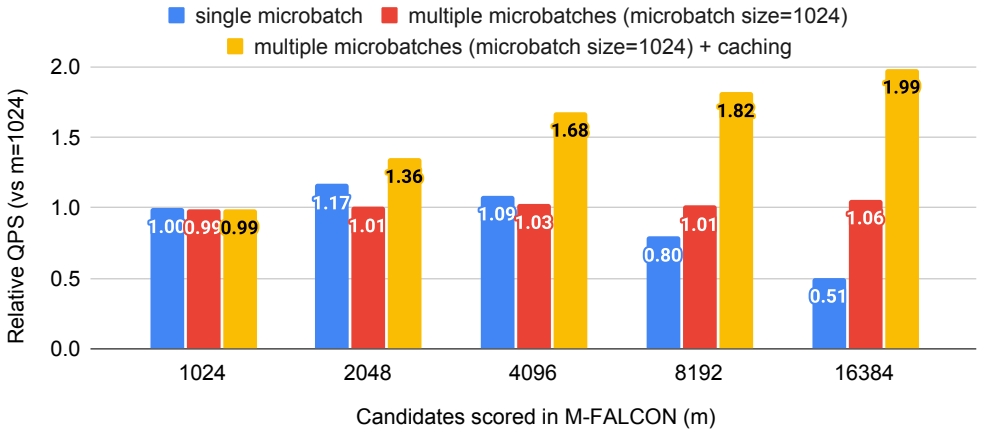


Figure 13. 端到端推理吞吐量：基于 FLOPs 为 285 倍的 GR 模型，M-FALCON 在大批量设置下的吞吐量扩展，其中 m（排序候选项总数）从 1024 到 16384，且 $b_m = 1024$。

Symbol	Description
$\Psi_k(t_j)$	The k-th training example (k is ordered globally) emitted by the feature logging system at time t_j. In a typical DLRM recommendation system, after the user consumes some content Φ_i (by responding with an action a_i such as skip, video completion and share), the feature logging system joins the tuple (Φ_i, a_i) with the features used to rank Φ_i, and *emits* $(\Phi_i, a_i,$ features for $\Phi_i)$ as a training example $\Psi_k(t_j)$. As discussed in Section 2.3, DLRMs and GRs deal with different numbers of training examples, with the number of examples in GRs typically being 1-2 orders of magnitude smaller.
n_c $(n_{c,i})$	Number of contents that user has interacted with (of user/sample i).
$\Phi_0, \ldots, \Phi_{n_c-1}$	List of contents that a user has interacted with, in the context of a recommendation system.
$a_0, \ldots, a_{n_c-1}$	List of user actions corresponding to Φ_is. When all predicted events are binary, each action can be considered a multi-hot vector over (atomic) events such as like, share, comment, image view, video initialization, video completion, hide, etc.
E, F	Categorical features in DLRMs, in Figure 2. $E_0, E_1, \ldots, E_7, E_8$, and $F_0, F_1, \ldots, F_7$ represent transformations of $(\Phi_0, a_0, t_0), \ldots, (\Phi_{n_c-1}, a_{n_c-1}, t_{n_c-1})$ obtained at various points in time via feature extraction (e.g., most recent 10 liked images, most similar 50 urls that the user clicked on in the past compared to the current candidate, etc.). "merge & sequentialize" denotes the (virtual) reverse process of obtaining the raw engagement series $(\Phi_0, a_0, t_0), \ldots, (\Phi_{n_c-1}, a_{n_c-1}, t_{n_c-1})$.
G, H	Categorical features in DLRMs, in Figure 2 that are not related to user-content engagements. These features (e.g., demographics or followed creators) are merged into the main time series (list of contents user engaged with, e.g., $\Phi_0, a_0, \ldots, \Phi_{n_c-1}, a_{n_c-1}$), as discussed in Section 2.1 and illustrated in Figure 2.
n (n_i)	Number of tokens in the sequential transduction task (of user/sample i). While $O(n) = O(n_c)$, n can differ from n_c even without any non-interaction related categorical features; see e.g., Table 1.
$x_0, \ldots, x_{n-1}$	List of input tokens in the sequential transduction task.
$y_0, \ldots, y_{n-1}$	List of output tokens in the sequential transduction task.
$t_0, \ldots, t_{n-1}$	List of timestamps corresponding to when $x_0, \ldots, x_{n-1}$ were observed.
$\mathbb{X}$, $\mathbb{X}_c$	Vocabulary of all input/output tokens ($\mathbb{X}$) and its content subset ($\mathbb{X}_c$).
N, N_c	$\max_i n_i$, $\max_i n_{c,i}$.
u_t	User representation at time t.
$s_u(n_i)$, $\hat{s}_u(n_i)$	Sampling rate for user i, used in generative training (Section 2.3).
d	Model dimension (embedding dimension).
d_{qk}	Attention dimension size in HSTU and Transformers. This applies to $Q(X)$ and $K(X)$ in Equation (1).
d_v	Value dimension size in HSTU. For Transformers, we typically have $d_{qk} = d_v$.
d_{ff}	Hidden dimension size in pointwise feedforward layers of Transformers. HSTU does not utilize feedforward layers; see $U(X)$ below.
h	Number of attention heads.
l	Number of layers in HSTU. For Transformers, attention and pointwise feedforward layers together constitute a layer.

Table 8. 符号表（下页续）。

Symbol	Description
X	Input to an HSTU layer. In standard terminology (before batching), $X \in \mathbb{R}^{N \times d}$ assuming we have a input sequence containing N tokens.
$Q(X), K(X), V(X)$	Query, key, value in HSTU obtained for a given input X based on Equation (1). The definition is similar to Q, K, and V in standard Transformers. $Q(X), K(X) \in \mathbb{R}^{h \times N \times d_{qk}}$, and $V(X) \in \mathbb{R}^{h \times N \times d_v}$.
$U(X)$	HSTU uses $U(X)$ to "gate" attention-pooled values ($V(X)$) in Equation (3), which together with $f_2(\cdot)$, enables HSTU to avoid feedforward layers altogether. $U(X) \in \mathbb{R}^{h \times N \times d_v}$.
$A(X)$	Attention tensor obtained for input X. $A(X) \in \mathbb{R}^{h \times N \times N}$.
$Y(X)$	Output of a HSTU layer obtained for the input X. $Y(X) \in \mathbb{R}^d$.
Split$(\cdot)$	The operation that splits a tensor into chunks. $\phi_1(f_1(X))) \in \mathbb{R}^{N \times (2hd_{qk}+2hd_v)}$ in Equation (1); we obtain $U(X)$, $V(X)$ (both of shape $h \times N \times d_v$), $Q(X)$, $K(X)$ (both of shape $h \times N \times d_{qk}$) by splitting the larger tensor (and permuting dimensions) with $U(X), V(X), Q(X), K(X) = \text{Split}(\phi_1(f_1(X)))$.
$\text{rab}^{p,t}$	relative attention bias that incorporates both positional (Raffel et al., 2020) and temporal information (based on the time when the tokens are observed, $t_0, \ldots, t_{n-1}$; one possible implementation is to apply some bucketization function to $(t_j - t_i)$ for (i, j)). In practice, we share $\text{rab}^{p,t}$ across different attention heads within a layer, hence $\text{rab}^{p,t} \in \mathbb{R}^{1 \times N \times N}$.
α	Parameter controlling sparsity in the *Stochastic Length* algorithm used in HSTU (Section 3.2).
R	Register size on GPUs, in the context of the HSTU algorithm discussed in Section 3.2.
m	Number of candidates considered in a recommendation system's ranking stage.
b_m	Microbatch size, in the M-FALCON algorithm discussed in Section 3.4.

Table 9. 符号表（续）

	Input for target item i	**Expected output for target item i**	**Architecture**	**Training Procedure**
GRs	$\Phi_0, a_0, \Phi_1, a_1, \ldots, \Phi_i$	a_i **(target-aware)**	Self-attention (HSTU)	**Causal autoregressive (streaming/single-pass)**
GRU4Rec SASRec	$\Phi_0, \Phi_1, \ldots, \Phi_{i-1}$	Φ_i	RNNs (GRUs) Self-attention (Transformers)	**Causal autoregressive** (multi-pass)
BERT4Rec S3Rec	$\Phi_0, \Phi_1, \ldots, \Phi_{i-1}$ (at inference time)	Φ_i	Self-attention (Transformers)	Sequential multi-pass [6]
DIN BST TWIN TransAct	$\Phi_0, \Phi_1, \ldots, \Phi_i$ $(\Phi_0, a_0), \ldots, (\Phi_{i-1}, a_{i-1}), \Phi_i$	a_i (**target aware**, implicitly as part of DLRMs)	Pairwise attention Self-attention (Transformers) Two-stage pairwise attention Self-attention (Transformers)	Pointwise (**generally streaming/single pass**)

Table 10. 排序设置下，既有序列推荐器研究与 GR 的比较；为完整起见，也纳入了 DLRM。

Task	**Specification (Inputs / Outputs / Length)**	
Next action token (a_i) prediction	x_is	$\Phi_0, a_0, \Phi_1, a_1, \ldots, \Phi_{n_c-2}, a_{n_c-2}, \Phi_{n_c-1}, a_{n_c-1}$
	y_is	$a_0, \varnothing, a_1, \varnothing, \ldots, a_{n_c-2}, \varnothing, a_{n_c-1}, \varnothing$
	n	$2n_c$
Next content token (Φ_i) prediction	x_is	$\Phi_0, a_0, \Phi_1, a_1, \ldots, \Phi_{n_c-2}, a_{n_c-2}, \Phi_{n_c-1}, a_{n_c-1}$
	y_is	$\varnothing, \Phi_1, \varnothing, \Phi_2, \ldots, \varnothing, \Phi_{n_c-1}, \varnothing, \varnothing$
	n	$2n_c$

Table 11. 对 $p(\Phi_0, a_0, \ldots, \Phi_{n_c-1}, a_{n_c-1})$ 进行生成式建模。示意见 Figure 8。

	Method	HR@10	HR@50	HR@200	NDCG@10	NDCG@200
ML-1M	SASRec (2023)	.2853	.5474	.7528	.1603	.2498
	BERT4Rec	.2843 (-0.4%)	–	–	.1537 (-4.1%)	–
	GRU4Rec	.2811 (-1.5%)	–	–	.1648 (+2.8%)	–
	HSTU	.3097 (+8.6%)	.5754 (+5.1%)	.7716 (+2.5%)	.1720 (+7.3%)	.2606 (+4.3%)
	HSTU-large	**.3294 (+15.5%)**	**.5935 (+8.4%)**	**.7839 (+4.1%)**	**.1893 (+18.1%)**	**.2771 (+10.9%)**
ML-20M	SASRec (2023)	.2906	.5499	.7655	.1621	.2521
	BERT4Rec	.2816 (-3.4%)	–	–	.1703 (+5.1%)	–
	GRU4Rec	.2813 (-3.2%)	–	–	.1730 (+6.7%)	–
	HSTU	.3252 (+11.9%)	.5885 (+7.0%)	.7943 (+3.8%)	.1878 (+15.9%)	.2774 (+10.0%)
	HSTU-large	**.3567 (+22.8%)**	**.6149 (+11.8%)**	**.8076 (+5.5%)**	**.2106 (+30.0%)**	**.2971 (+17.9%)**
Books	SASRec (2023)	.0292	.0729	.1400	.0156	.0350
	HSTU	.0404 (+38.4%)	.0943 (+29.5%)	.1710 (+22.1%)	.0219 (+40.6%)	.0450 (+28.6%)
	HSTU-large	**.0469 (+60.6%)**	**.1066 (+46.2%)**	**.1876 (+33.9%)**	**.0257 (+65.8%)**	**.0508 (+45.1%)**

Table 12. 传统序列推荐器设置（多遍、完全打乱）下各方法在公开数据集上的评估。与 Table 4 相比，为完整起见，额外纳入了 GRU4Rec 和 BERT4Rec 两个基线。

Metric Name	Selection Type		
	Greedy	Weighted	Random
Main Engagement Metric (NE)	0.495	0.494	0.495
Main Consumption Metric (NE)	0.792	0.789	0.791

Table 13. *随机长度*的子序列选择方法对模型质量的影响比较，以归一化熵（NE）衡量。

Alpha	Max Sequence Length							
	1,024		2,048		4,096		8,192	
	sparsity	s2	sparsity	s2	sparsity	s2	sparsity	s2
1.6	71.5%	89.4%	75.8%	92.3%	79.4%	94.7%	83.8%	97.3%
1.7	57.3%	77.6%	60.6%	79.8%	67.3%	86.6%	74.5%	93.3%
1.8	37.5%	56.2%	42.6%	62.1%	51.9%	74.2%	62.6%	85.5%
1.9	15.0%	25.2%	17.7%	29.0%	29.6%	47.5%	57.8%	80.9%
2.0	1.2%	1.7%	2.5%	3.5%	18.9%	30.8%	57.6%	80.6%

Table 14. 在 60 天用户互动历史上，*随机长度*（SL）对序列稀疏率的影响。

Alpha	Max Sequence Length							
	1,024		2,048		4,096		8,192	
	sparsity	s2	sparsity	s2	sparsity	s2	sparsity	s2
1.6	68.0%	85.0%	74.6%	90.8%	78.6%	93.5%	83.5%	97.3%
1.7	56.3%	76.1%	61.2%	80.6%	67.5%	87.0%	74.3%	93.3%
1.8	38.9%	58.3%	42.0%	61.3%	50.4%	72.4%	61.0%	84.4%
1.9	16.2%	27.3%	17.3%	28.6%	27.2%	44.4%	54.3%	77.8%
2.0	0.9%	1.2%	1.6%	2.1%	13.5%	22.5%	54.0%	77.4%

Table 15. 在 90 天用户互动历史上，*随机长度*（SL）对序列稀疏率的影响。

Evaluation Strategy	**Average NE Difference vs Full Sequence Baseline**		
	Model Type	2048 / 52% Sparsity	4096 / 75% Sparsity
Zero-shot	HSTU (Raffel et al., 2020)	6.46%	10.35%
	HSTU-RoPE (Peng et al., 2024)	7.51%	11.27%
Fine-tune	HSTU (Raffel et al., 2020)	1.92%	2.21%
	HSTU-RoPE (Peng et al., 2024)	1.61%	2.19%
Stochastic Length (SL)	HSTU	**0.098**%	**0.64**%

Table 16. 随机长度（SL）与现有长度外推方法的比较。